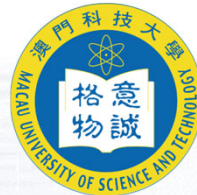


2025 Autonomous Driving Seminar

Artificial Intelligence Techniques in Autonomous Driving



Dagang Li

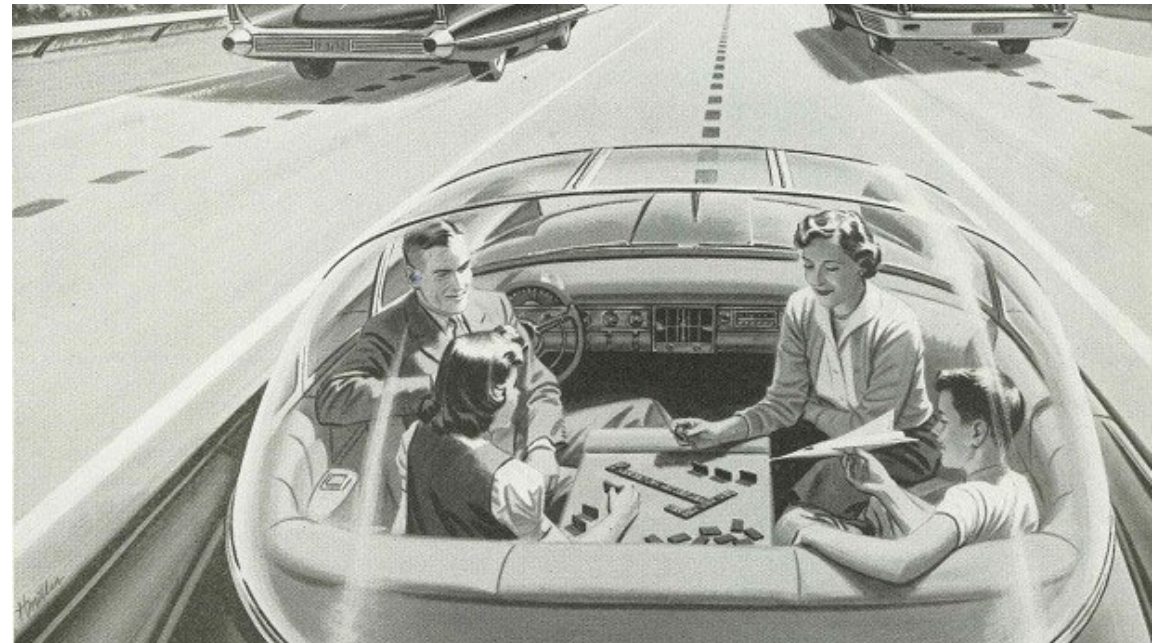
School of Computer Science and Engineering

Content

- 1. Background
- 2. Autonomous Driving Approaches
- 3. Modular Pipeline
- 4. End-to-end Learning
- 5. Direct Perception
- 6. Reinforcement Learning

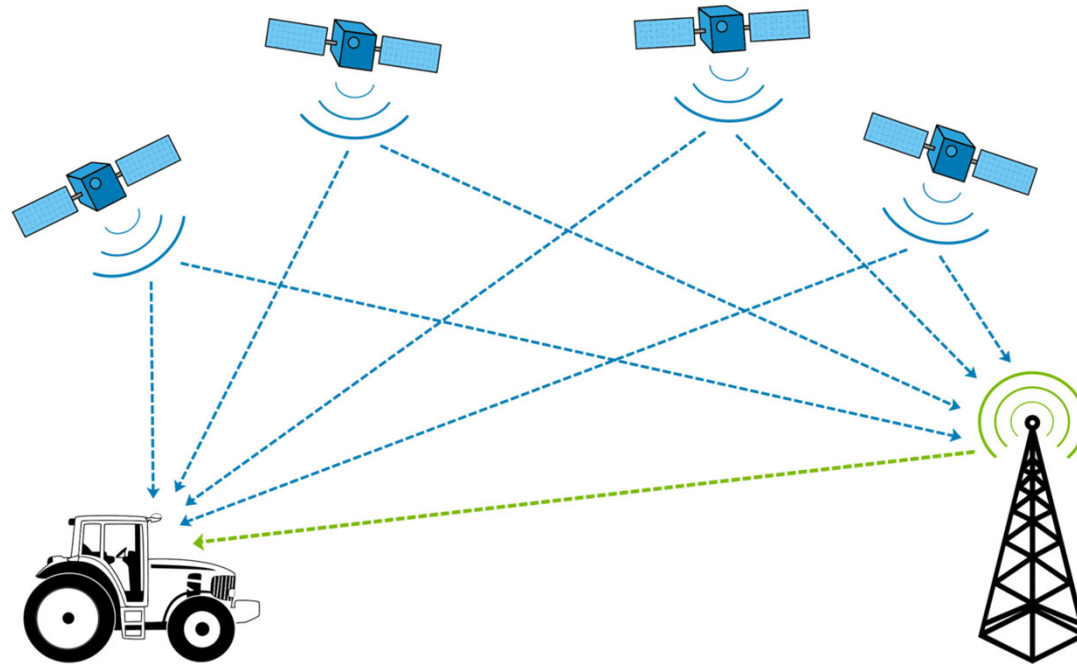
Autonomous Driving / Self-driving / Automated Driving

- Human Dream: relaxed journey
- No-driver freight transportation
- Eliminate human errors



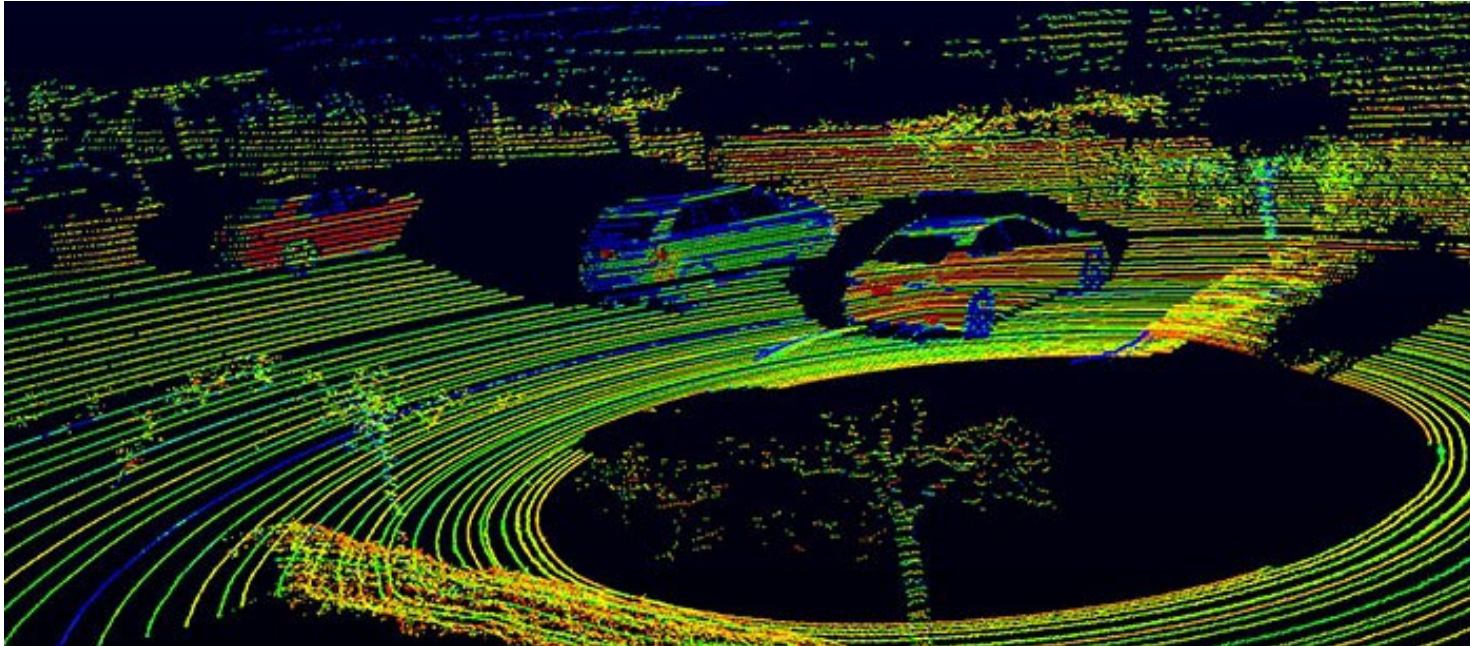
- Mobility for elderly and people with disabilities
- New ways of public transportation
- Reduce number of cars

2000: First Technological Revolution: GPS, IMUs & Maps



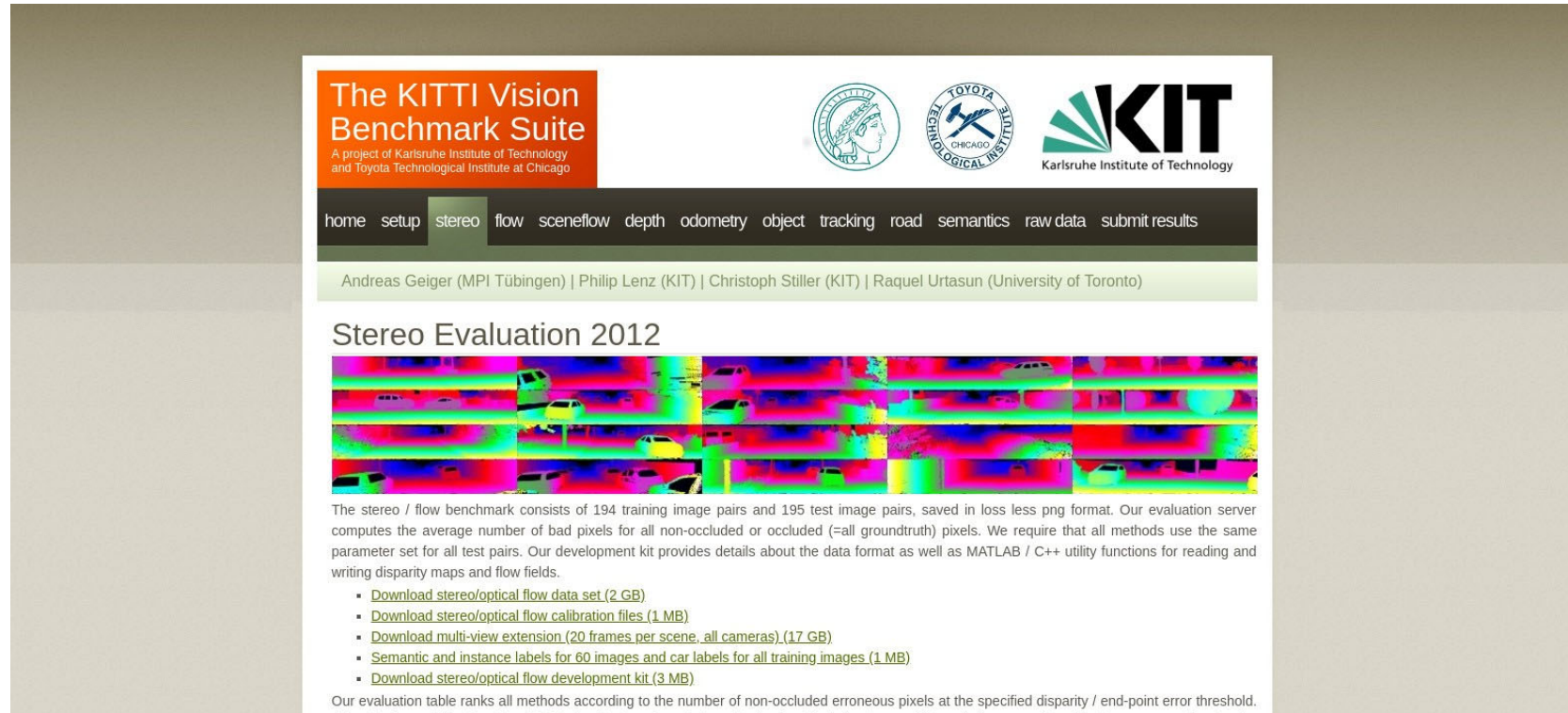
- NAVSTAR GPS available with 1 meter accuracy, IMUs improve up to 5 cm
- Navigation systems and road maps available
- Accurate self-localization and ego-motion estimation algorithms

2006: Second Technological Revolution: Lidars & High-res Sensors



- High-resolution Lidar
- Camera systems with increasing resolution
- Accurate 3D reconstruction, 3D detection & 3D localization

2012: Third Technological Revolution: Deep Learning



- Representation learning boosts in accuracy across tasks and benchmarks

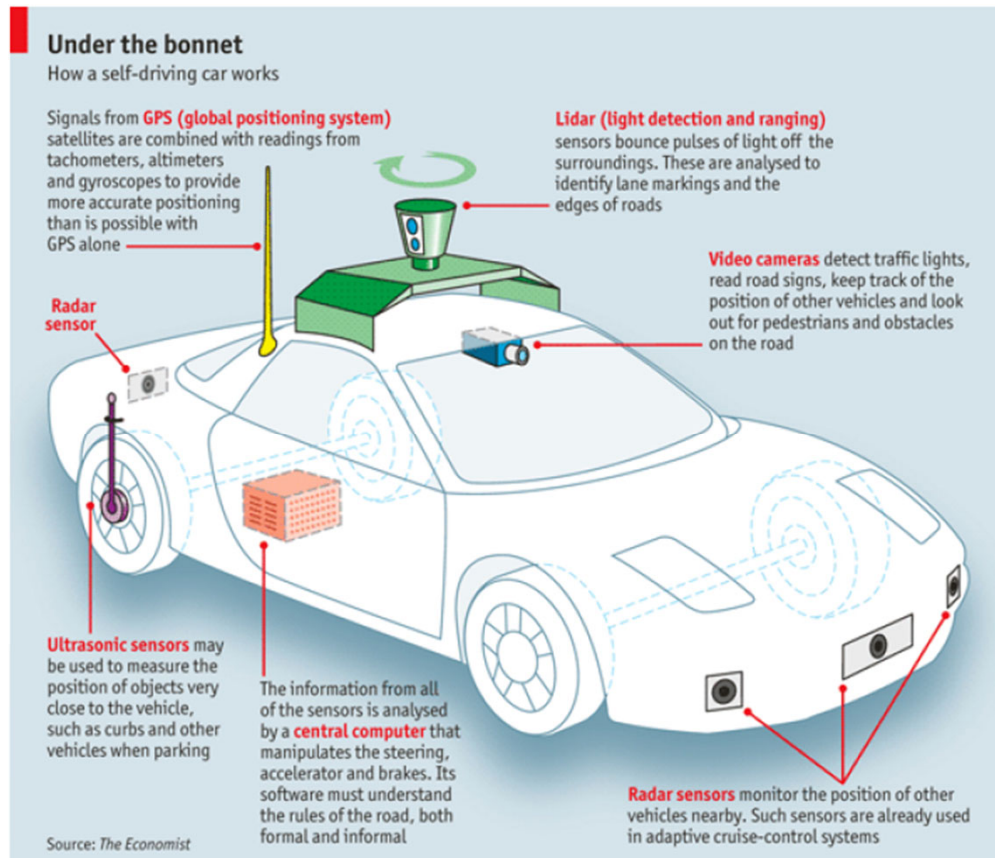
2014: Society of Automotive Engineers: SAE Levels of Autonomy

			Steering, acceleration / deceleration	Monitoring of driving environment	Fallback when automation fails	Automated system is in control
Human driver monitors the road	0	No Automation (1885 to 1999)	 Eyes on Hands on			Never
	1	Driver Assistance (2000 to 2009)	 Eyes on Hands on			Present in some driving modes
	2	Partial Automation (2000 until today)	 Temporary hands off			Present in some driving modes
Automated driving monitors the road	3	Conditional Automation (current stage)	 Temporary hands off			Present in some driving modes
	4	High Automation (estimate by 2025)	 Eyes off Hands off			Present in some driving modes
	5	Full Automation (estimate by 2050)	 Eyes off Hands off			

Evolution to Autonomous Driving

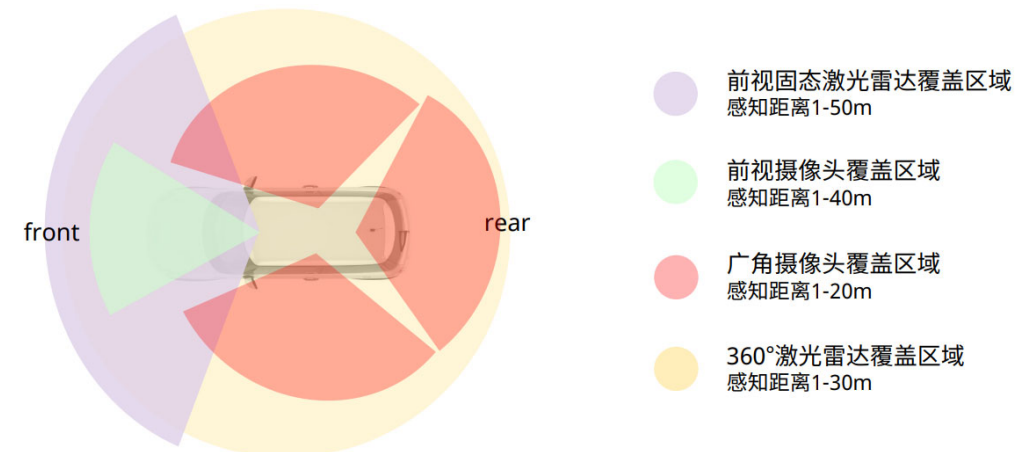
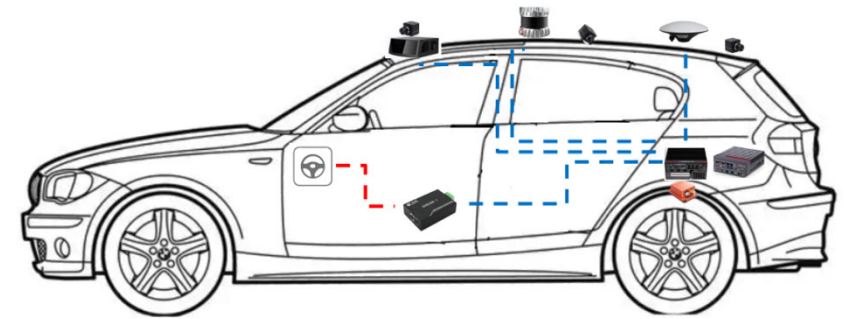
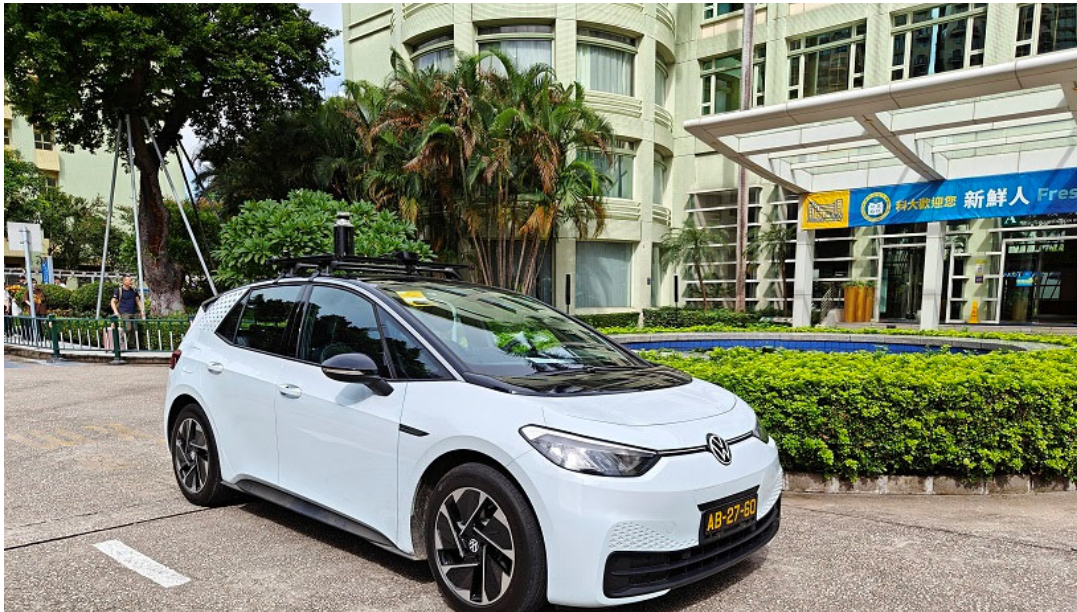
- Autonomous or nothing (Google, Apple, Uber, Huawei)
 - Very risky, only few companies can do this
 - Long term goals
- Introduce technology little by little (all car companies)
 - Car industry is very conservative
 - ADAS as intermediate goal
 - Sharp transition: how to maintain the driver engagement?

Basic Physical Ecosystem of an Autonomous Vehicle



- Global Positioning System (GPS)
- Light Detection and Ranging (LIDAR)
- Cameras (Video)
- Ultrasonic Sensors
- Central Computer
- Radar Sensors
- Dedicated Short-Range Communications-Based Receiver (not pictured)

MUST Open Intelligent Electric Vehicles Research Center (OIEV-RC)



Autonomous Driving Approaches

Autonomous Driving

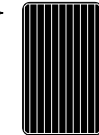
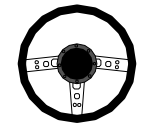
Sensory Input



Mapping Function



Steer



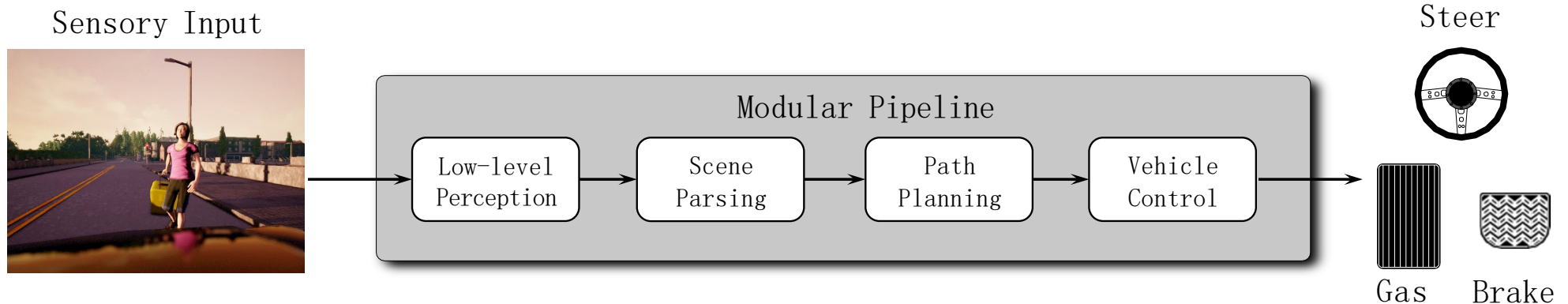
Gas

Brake

Dominating Paradigms:

- Modular Pipelines
- End-to-End Learning (Imitation Learning, Reinforcement Learning)
- Direct Perception

Autonomous Driving: Modular Pipeline



Pros:

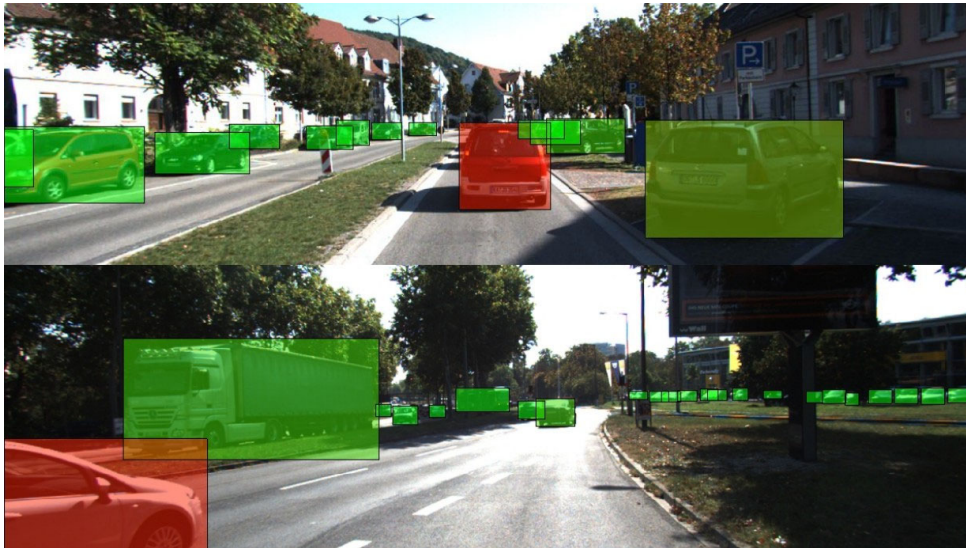
- Small components, easy to develop in parallel
- Interpretability

Cons:

- Piece-wise training (not jointly)
- Localization and planning heavily relies on HD maps

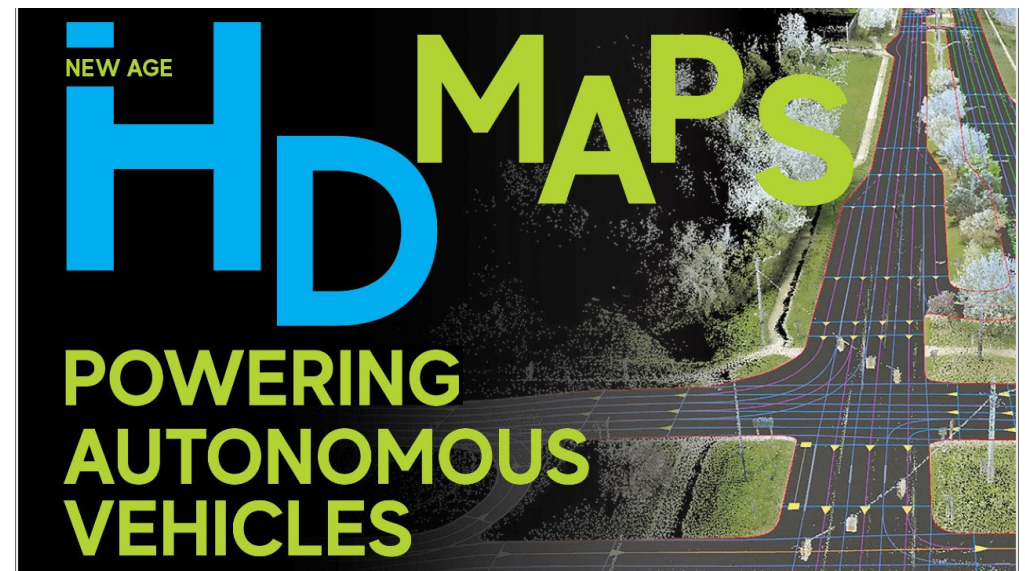
HD maps: Centimeter precision lanes, markings, traffic lights/signs, human annotated

Autonomous Driving: Modular Pipeline

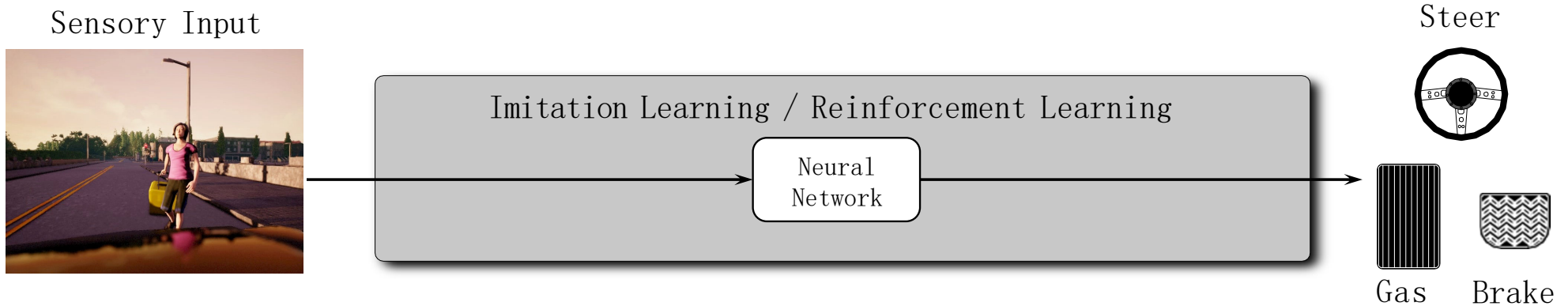


Piece-wise training difficult: not all objects are equally important!

HD Maps are expensive to create
(data collection & annotation effort)



Autonomous Driving: End-to-End Learning



Pros:

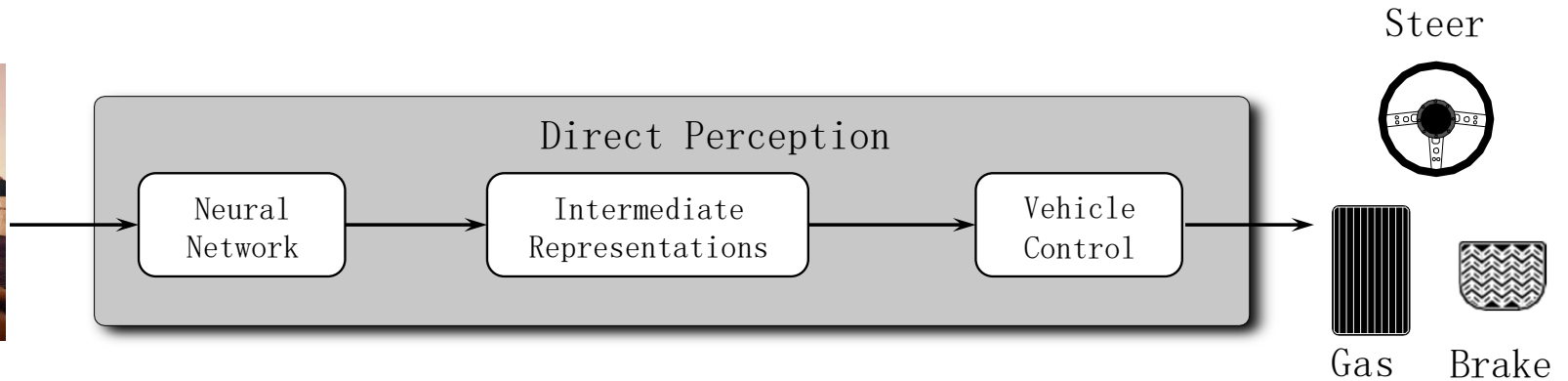
- End-to-end training
- Simple
- Cheap annotations

Cons:

- Training / Generalization
- Interpretability
- Data hungry

Autonomous Driving: Direct Perception

Sensory Input



Pros:

- Compact Representation
- Interpretability

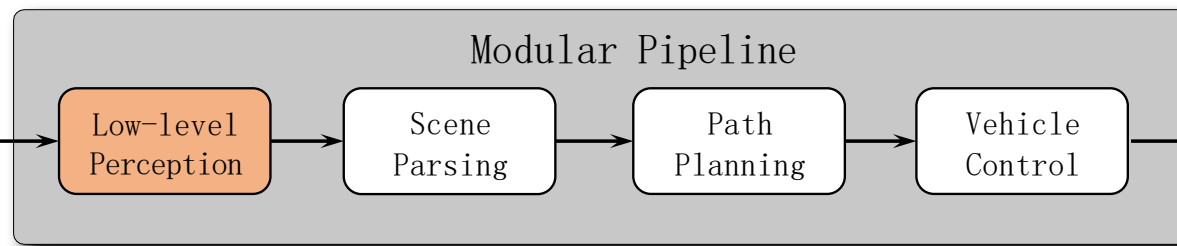
Cons:

- Control typically not learned jointly
- How to choose representations?

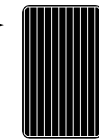
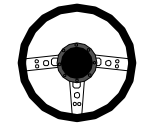
Autonomous Driving: Modular Pipeline

Autonomous Driving: Modular Pipeline

Sensory Input



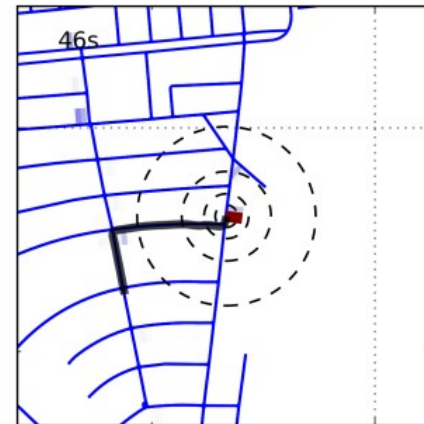
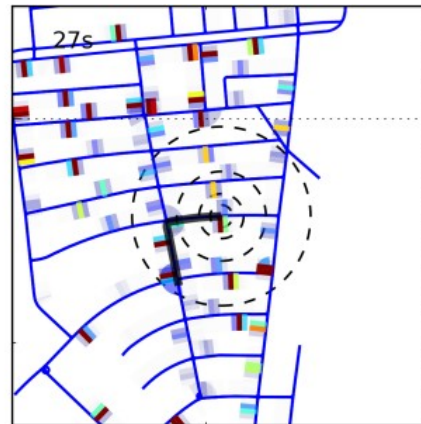
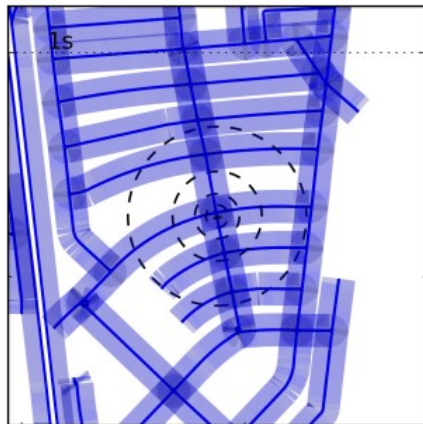
Steer



Gas

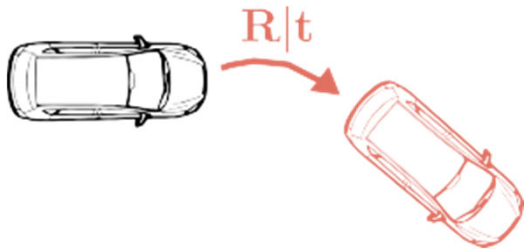


Brake

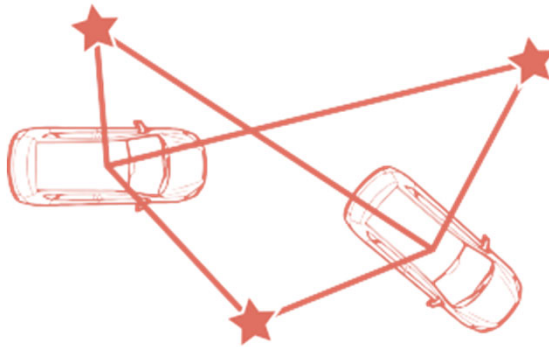


Ego-motion Estimation and Localization

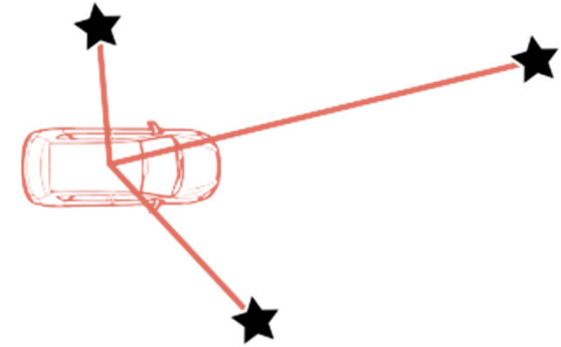
Self-driving cars operate in a dynamic environments and hence must be aware of their ego-motion (own motion) and the motion of other traffic participants.



Visual Odometry



SLAM

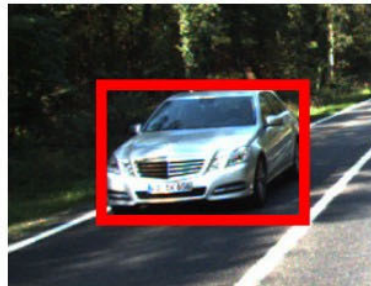
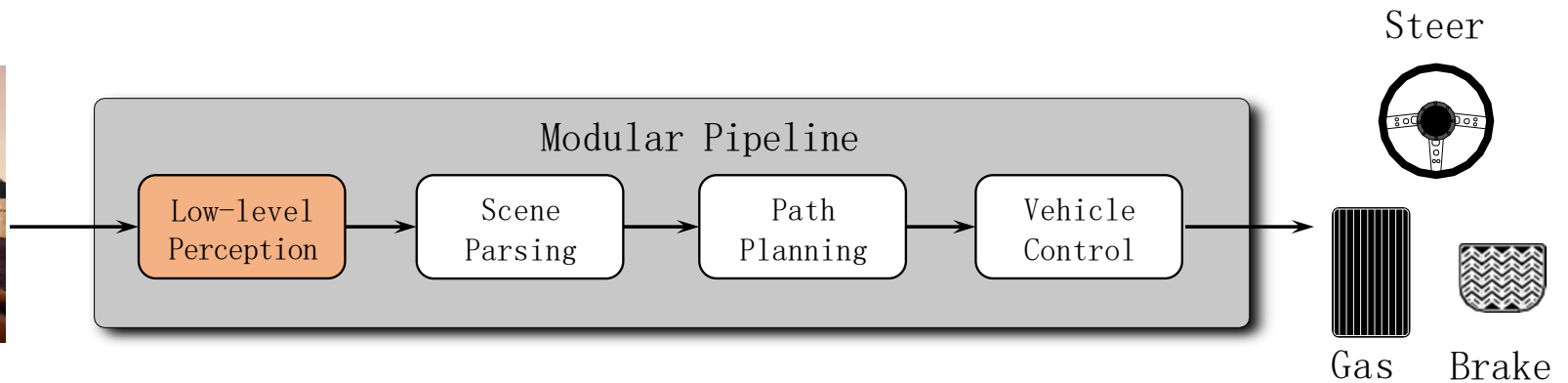


Localization

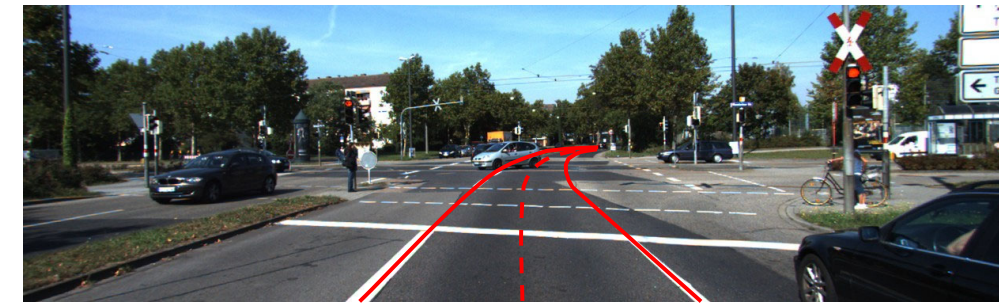
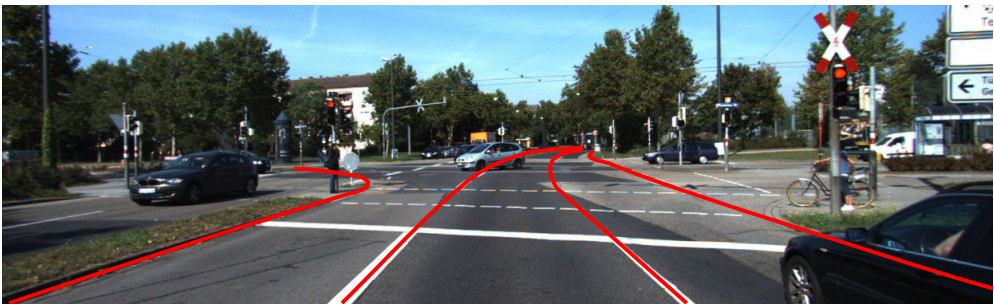
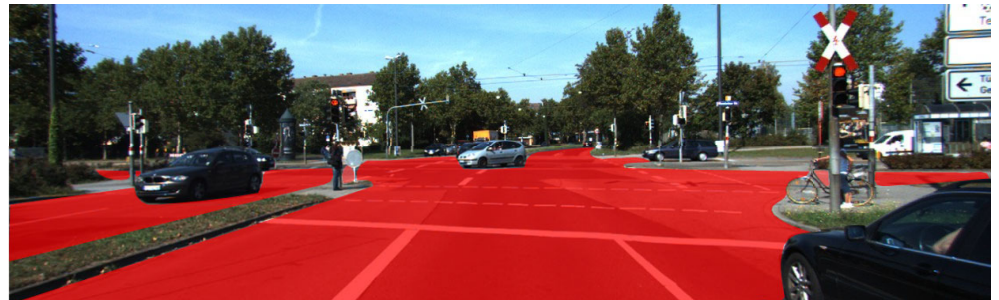
- **Visual odometry** refers to the estimation of **relative ego-motion** from images
- **SLAM algorithms** build a **map** and simultaneously **localize** in that map
- **Localization methods** find the **global pose** of a vehicle in a given map

Autonomous Driving: Modular Pipeline

Sensory Input

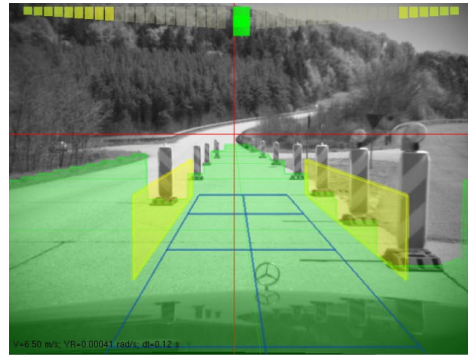


Road and Lane Detection

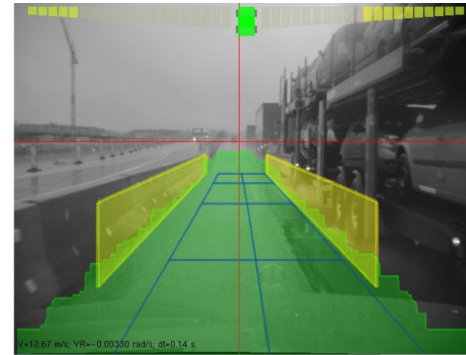


Occupancy Map

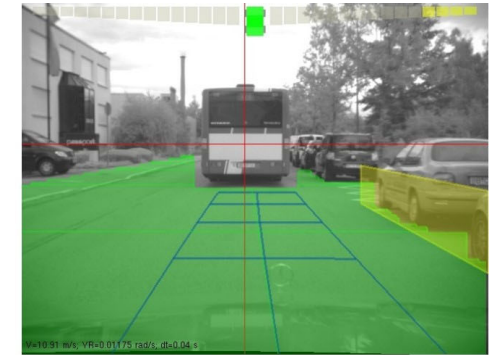
Height of obstacles
unknown / only
freespace estimation



(a) Highway.



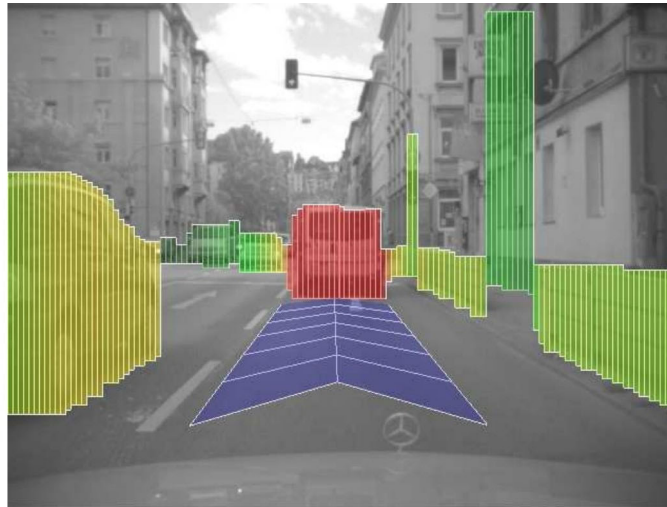
(b) Freeway.



(c) Downtown.



(a) Dense disparity image (SGM result)



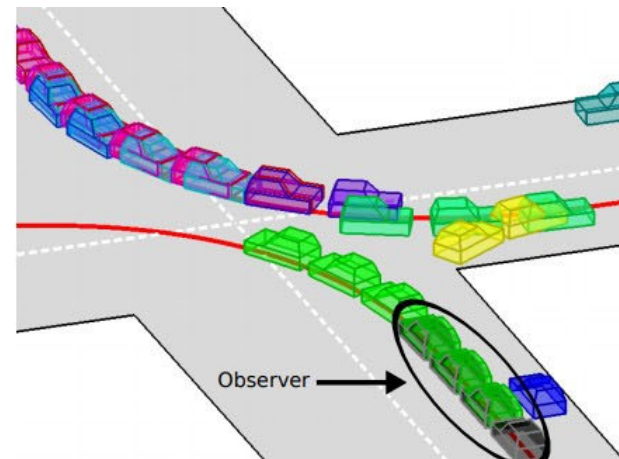
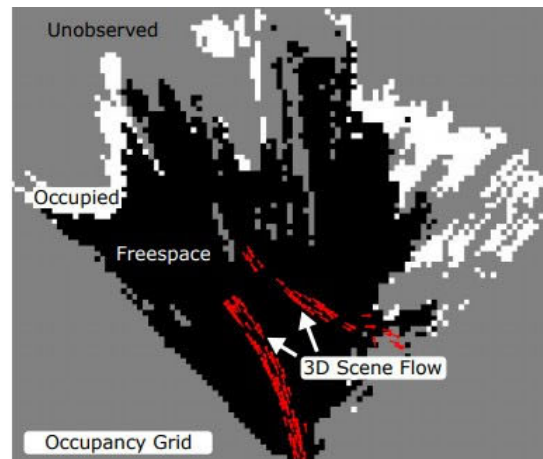
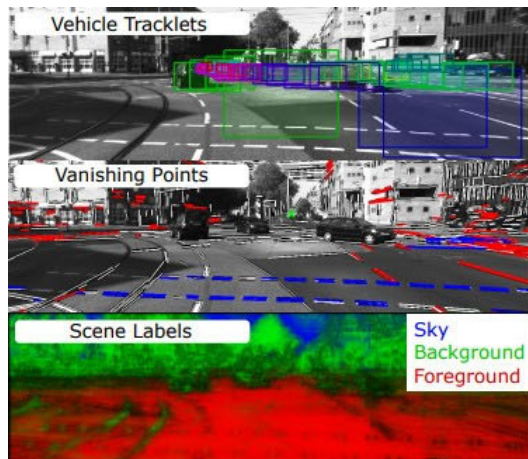
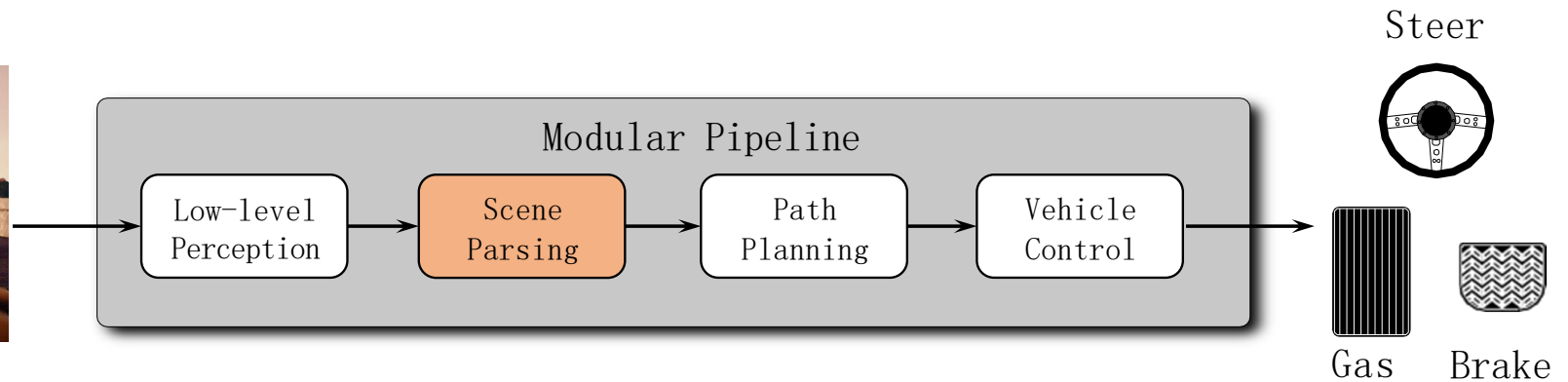
(b) Stixel representation

The Stixel World

“Stixel World” estimates
multiple stixels per image
column (i.e., segmentation)

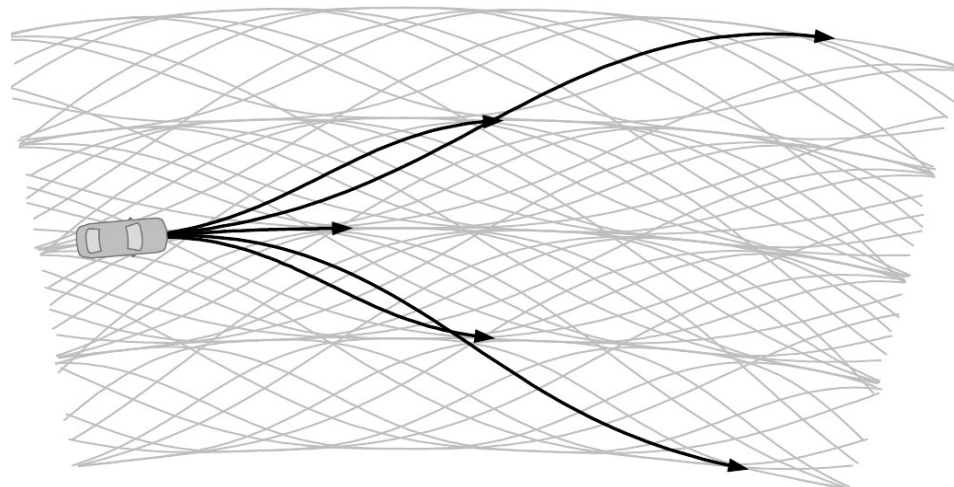
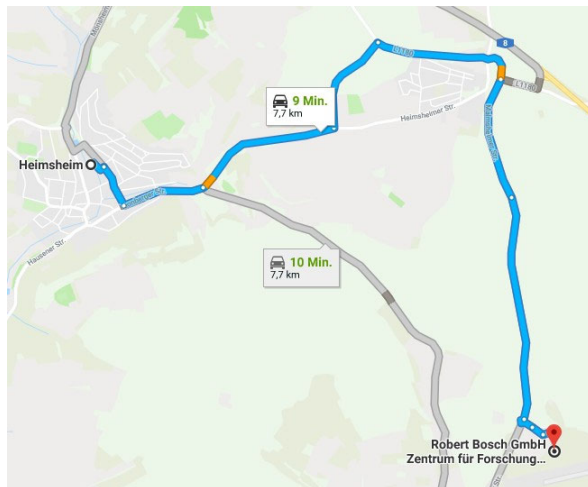
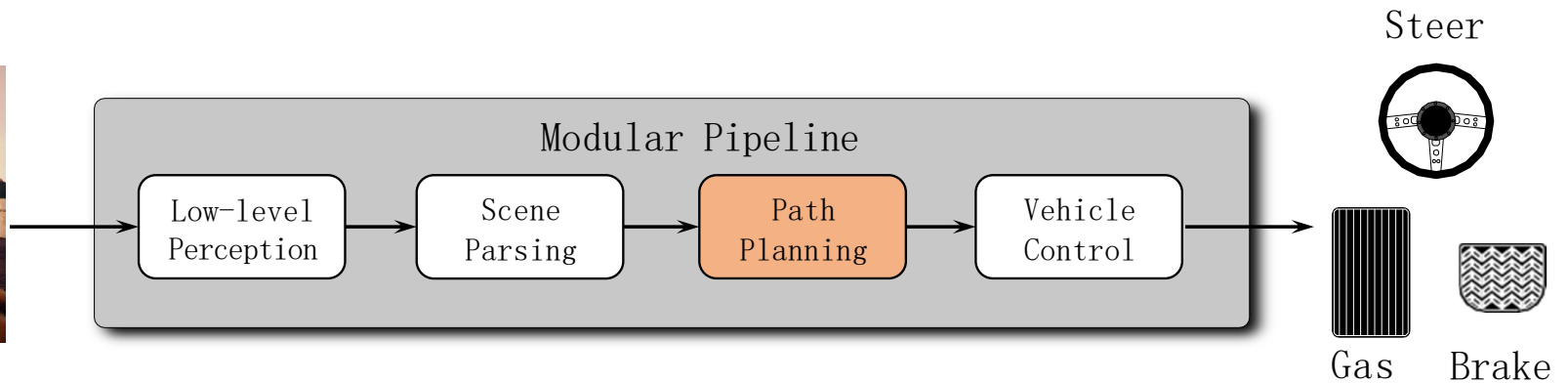
Autonomous Driving: Modular Pipeline

Sensory Input



Autonomous Driving: Modular Pipeline

Sensory Input



Planning and Decision Making

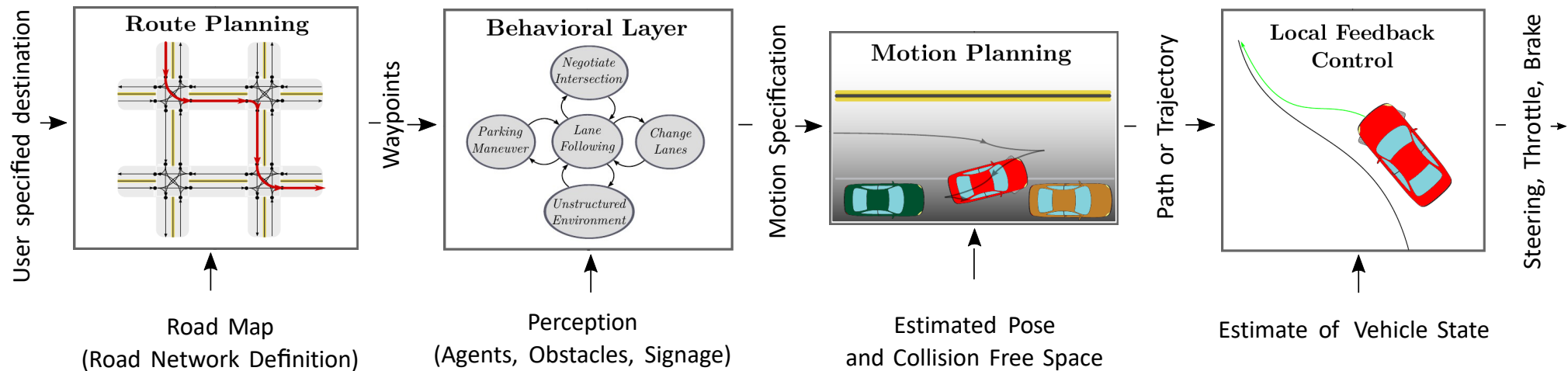
■ Problem Definition:

- Goal: find and follow path from current location to destination
- Take static infrastructure and dynamic objects into account
- Input: vehicle and environment state (via perception stack)
- Output: path or trajectory as input to vehicle controller

■ Challenges:

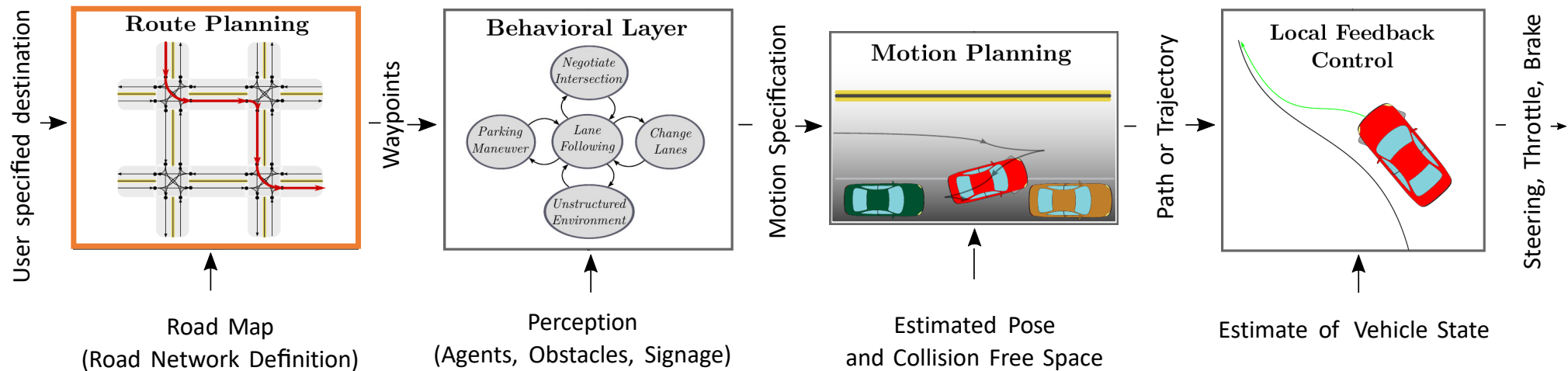
- Driving situations and behaviors are very complex
- Thus difficult to model as a single optimization problem

Planning and Decision Making



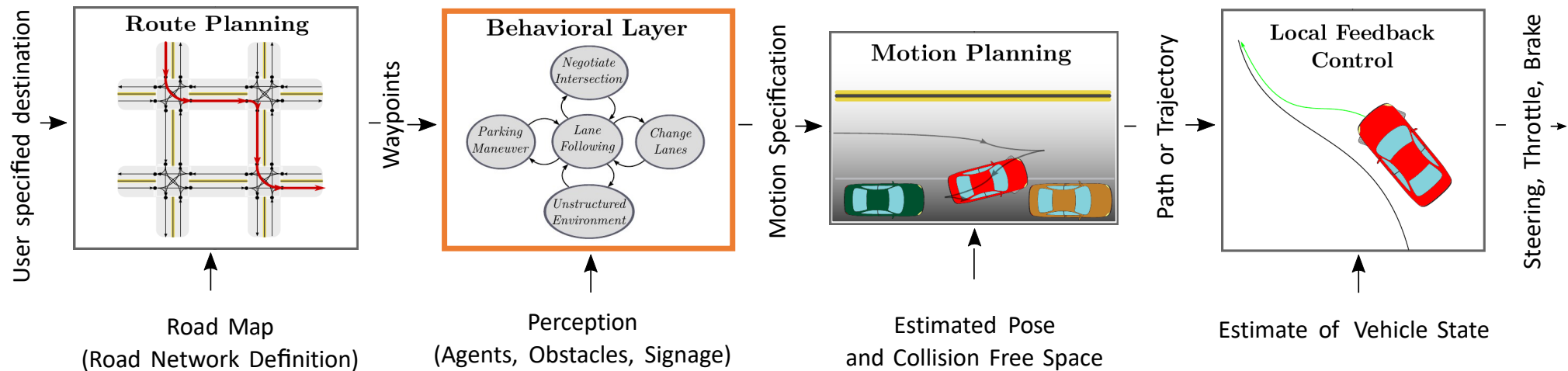
- Idea: Break planning problem into a **hierarchy of simpler problems**
- Each problem tailored to its scope and level of abstraction
- Earlier in this hierarchy means higher level of abstraction
- Each optimization problem will have constraints and objective functions

Step 1: Route Planning



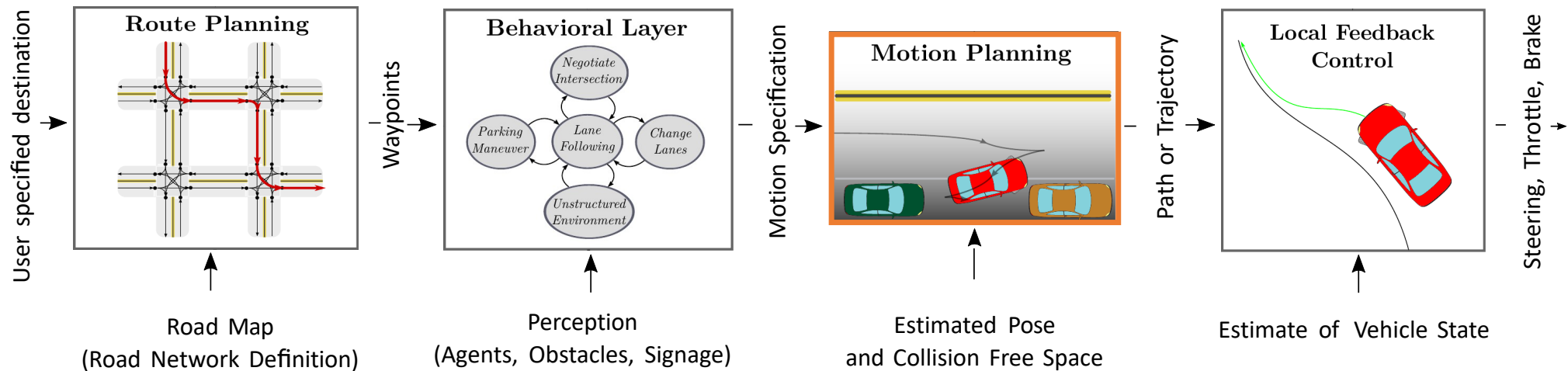
- Represent road network as directed graph
- Edge weights correspond to road segment length or travel time
- Problem translates into a minimum-cost graph network problem
- Inference algorithms: Dijkstra, A*, . . .

Step 2: Behavior Planning



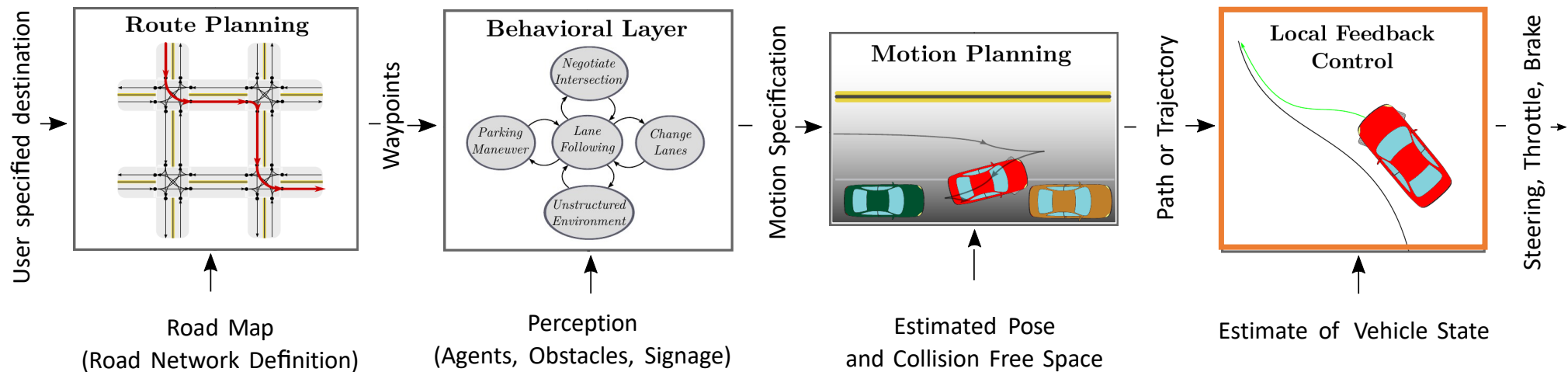
- Select driving behavior based on current vehicle/environment state
- E.g. at stop line: stop, observe other traffic participants, traverse
- Often modeled via finite state machines (transitions governed by perception)
- Can be modeled probabilistically, e.g., using Markov Decision Processes (MDPs)

Step 3: Motion Planning



- Find feasible, comfortable, safe and fast vehicle path/trajectory
- Exact solutions in most cases computationally intractable
- Thus often numerical approximations are used
- Approaches: variational methods, graph search, incremental tree-based

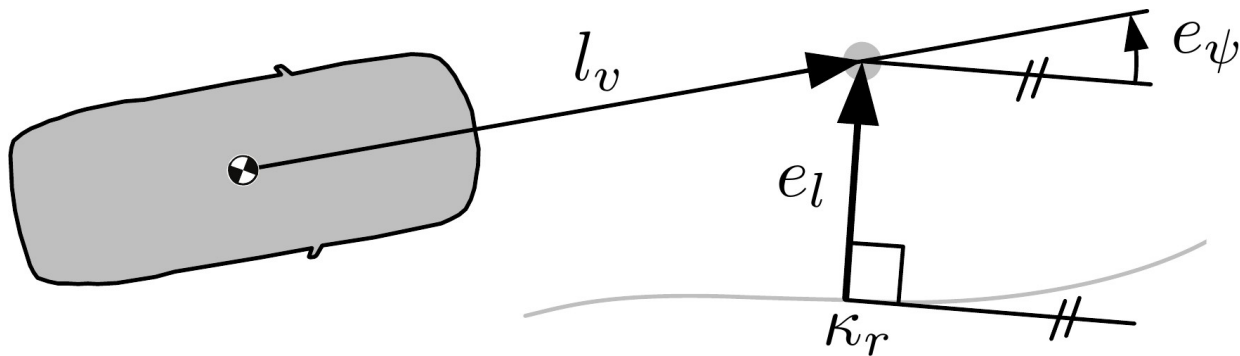
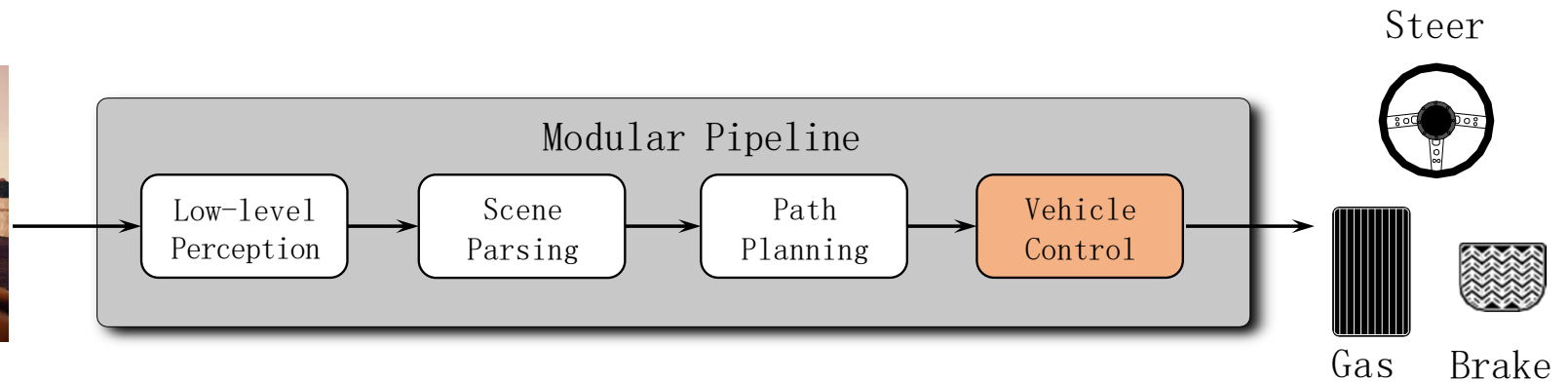
Step 4: Local Feedback Control



- Feedback controller executes the path/trajectory from the motion planner
- Corrects errors due to inaccuracies of the vehicle model
- Emphasis on robustness, stability and comfort
- Interact with underlying vehicle dynamics and control

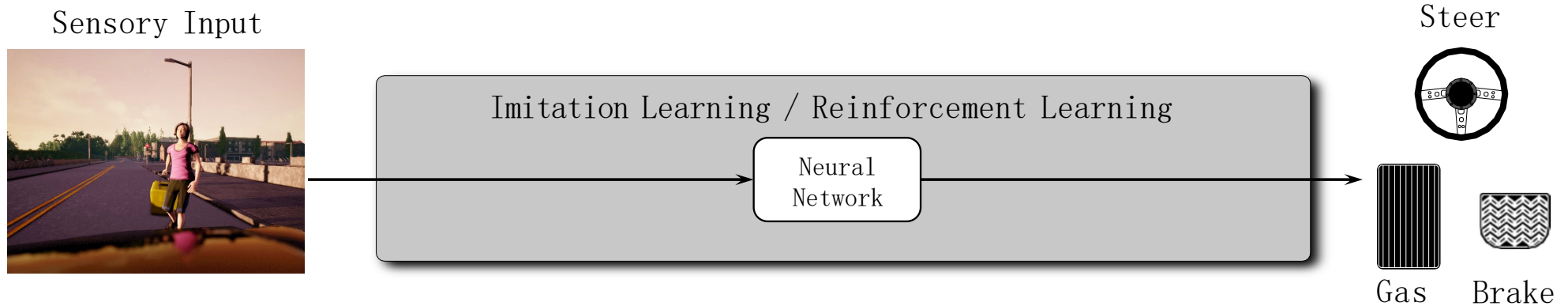
Autonomous Driving: Modular Pipeline

Sensory Input



Autonomous Driving: End-to-end Learning

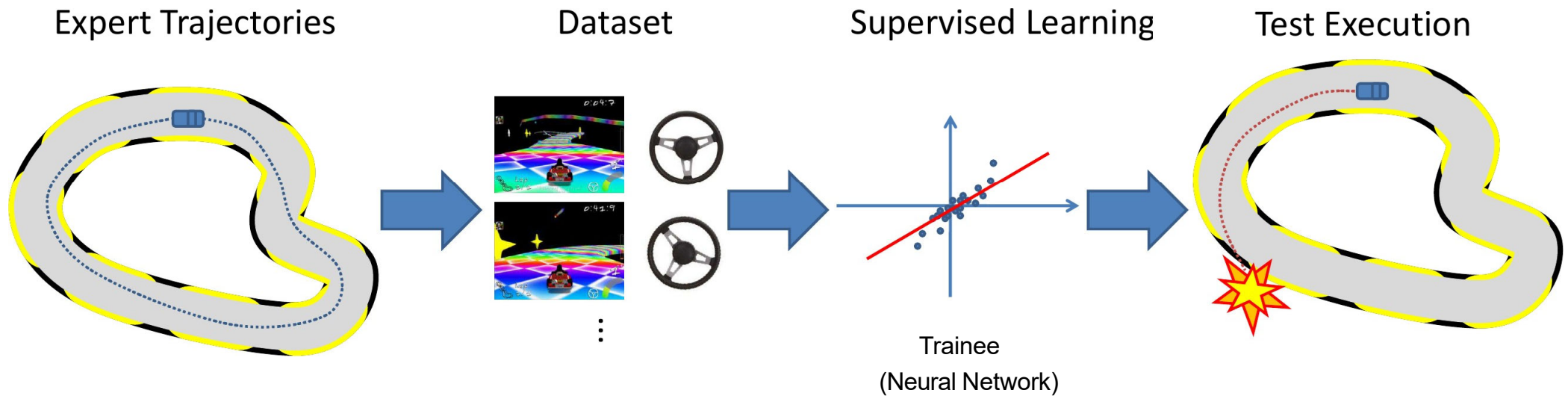
Autonomous Driving: End-to-End Learning



Idea:

- Generates ego-motion directly from the sensory input
- Perception, prediction and planning can be jointly trained and optimized towards the ultimate task
- Shared backbones increase computational efficiency
- Allow continuous learning to sense and act

Imitation Learning in a Nutshell



Hard coding policies is often difficult $\Rightarrow$ Rather use a data-driven approach!

- **Given:** demonstrations or demonstrator
- **Goal:** train a policy to mimic decision
- **Variants:** behavior cloning (this lecture), inverse optimal control, . . .

Challenges of Behavior Cloning

- Behavior cloning makes IID assumption
 - Next state is sampled from states observed during expert demonstration
 - Thus, next state is sampled independently from action predicted by current policy
- What if the trainee network makes a mistake?
 - Enters new states that haven't been observed before
 - New states not sampled from same (expert) distribution anymore
 - Cannot recover, catastrophic failure in the worst case
- What can we do to overcome this train/test distribution mismatch?

Conditional Imitation Learning



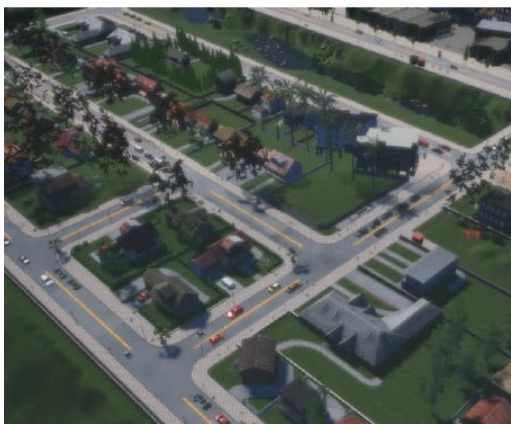
(a) Aerial view of test environment



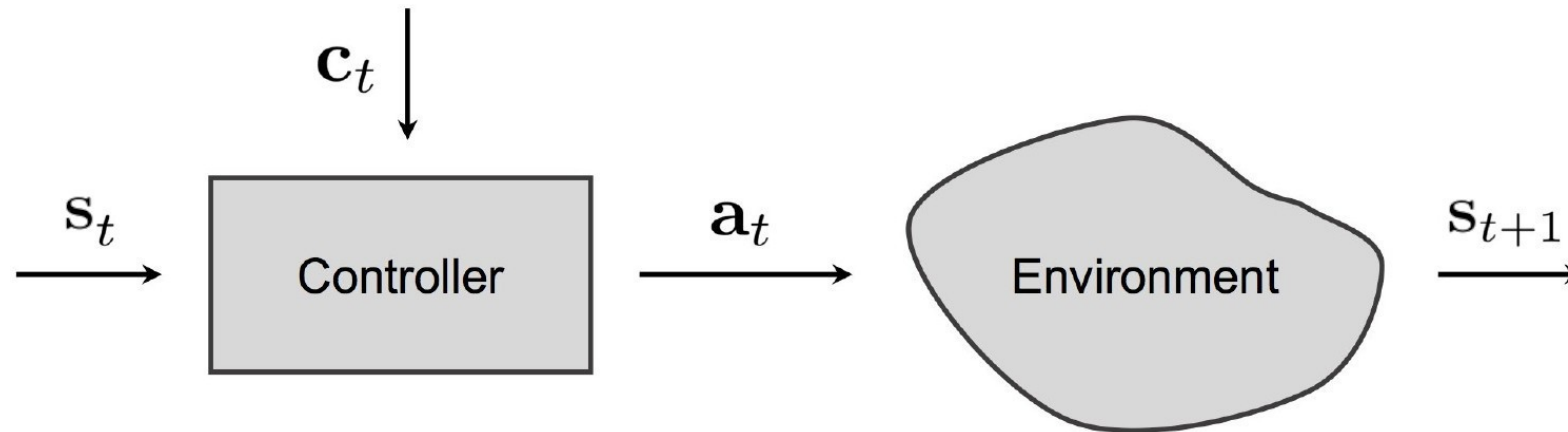
(b) Vision-based driving, view from onboard camera



(c) Side view of vehicle



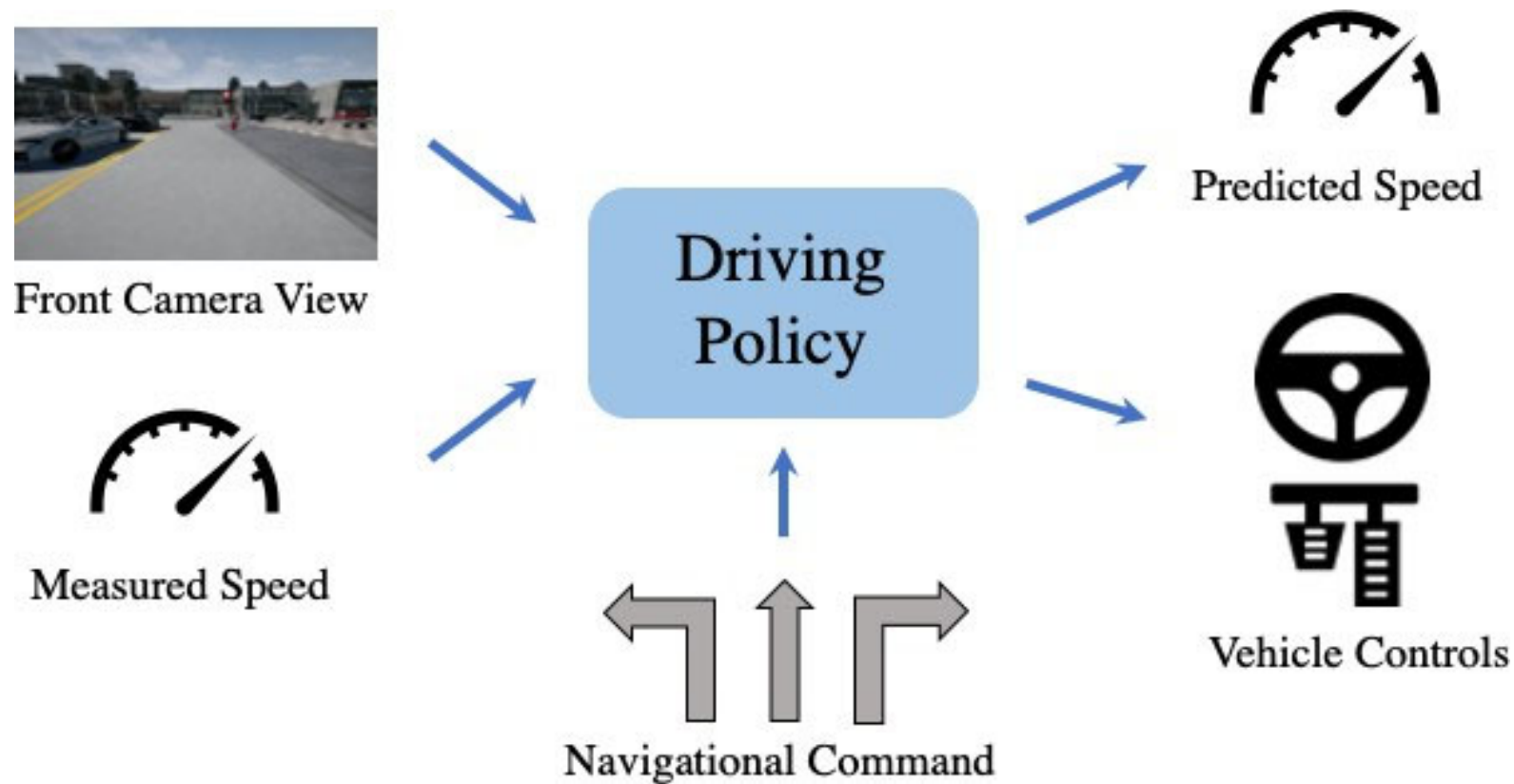
Conditional Imitation Learning



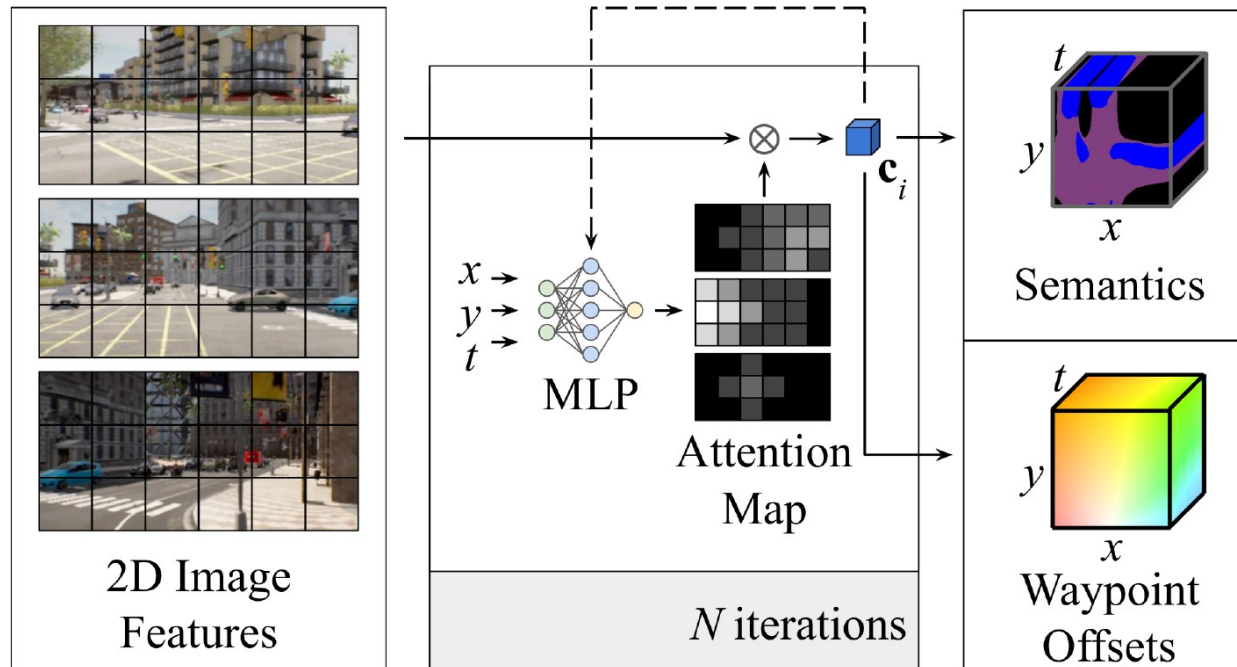
Idea:

- Condition controller on **navigation command** $c \in \{\text{left, right, straight}\}$
- High-level navigation command can be provided by consumer GPS, i.e., telling the vehicle to turn left/right or go straight at the next intersection
- This removes the task ambiguity induced by the environment
- State s_t : current image Action a_t : steering angle & acceleration

Conditional Imitation Learning



Neural Attention Fields



- An MLP iteratively **compresses** the high-dimensional input into a compact representation c_i based on a BEV query location as input
- The model predicts **waypoints** and auxiliary **semantics** which aids learning

Advantages and Challenges of Imitation Learning

■ Advantages of Imitation Learning:

- Easy to implement
- Cheap annotations (just driving while recording images and actions)
- Entire model trained end-to-end
- Conditioning removes ambiguity at intersections

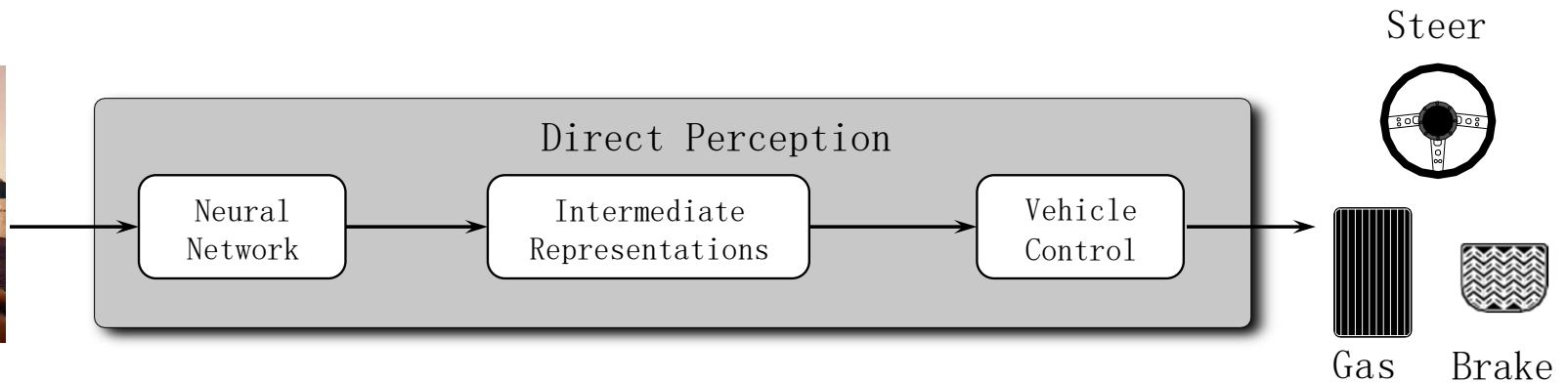
■ Challenges for Imitation Learning?

- Behavior cloning uses IID assumption which is violated in practice
- Direct mapping from images to control $\Rightarrow$ No long term planning
- No memory (can't remember speed signs, etc.)
- Mapping is difficult to interpret ("black box"), despite visualization techniques

Autonomous Driving: Direct Perception

Direct Perception

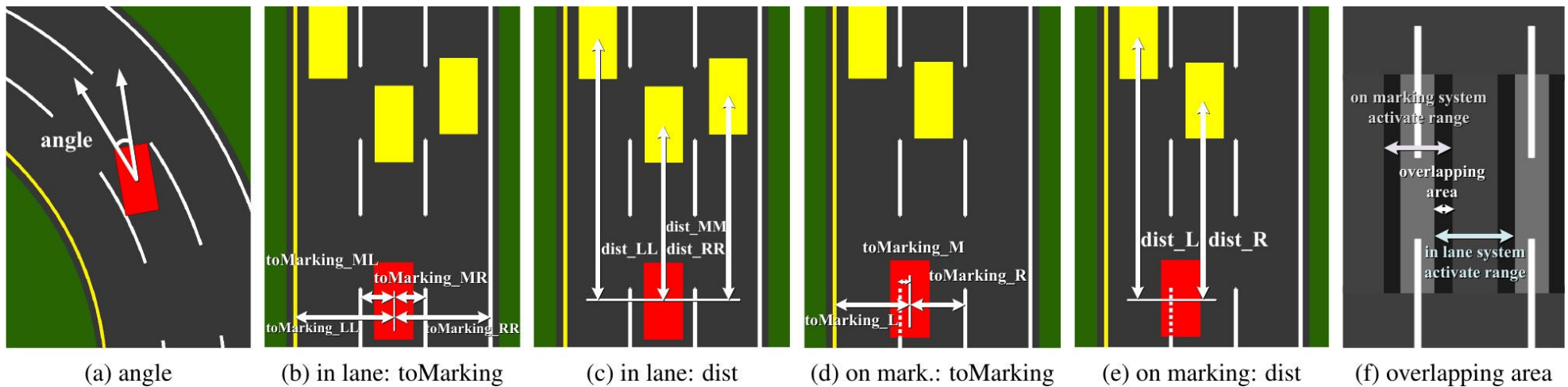
Sensory Input



Idea:

- **Hybrid model** between imitation learning and modular pipelines
- Learn to predict interpretable **low-dimensional** intermediate representation
- **Decouple** perception from planning and control
- Allows to exploit **classical** controllers or **learned** controllers (or hybrids)

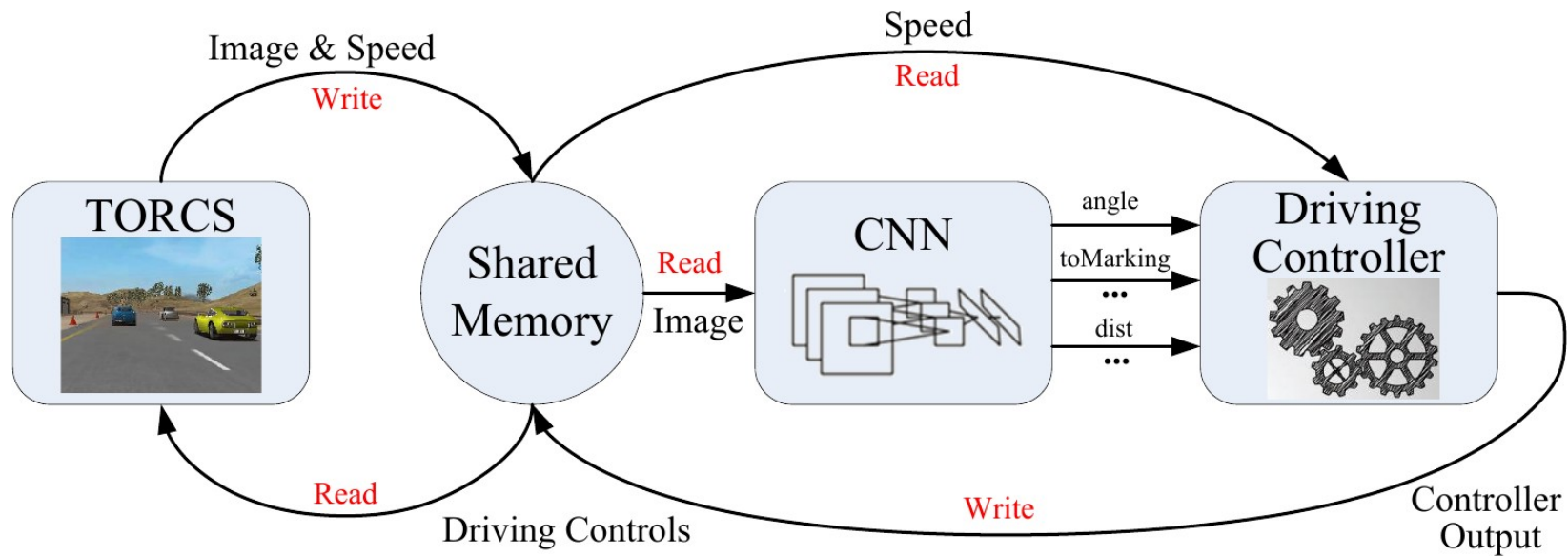
Direct Perception for Autonomous Driving



Affordances:

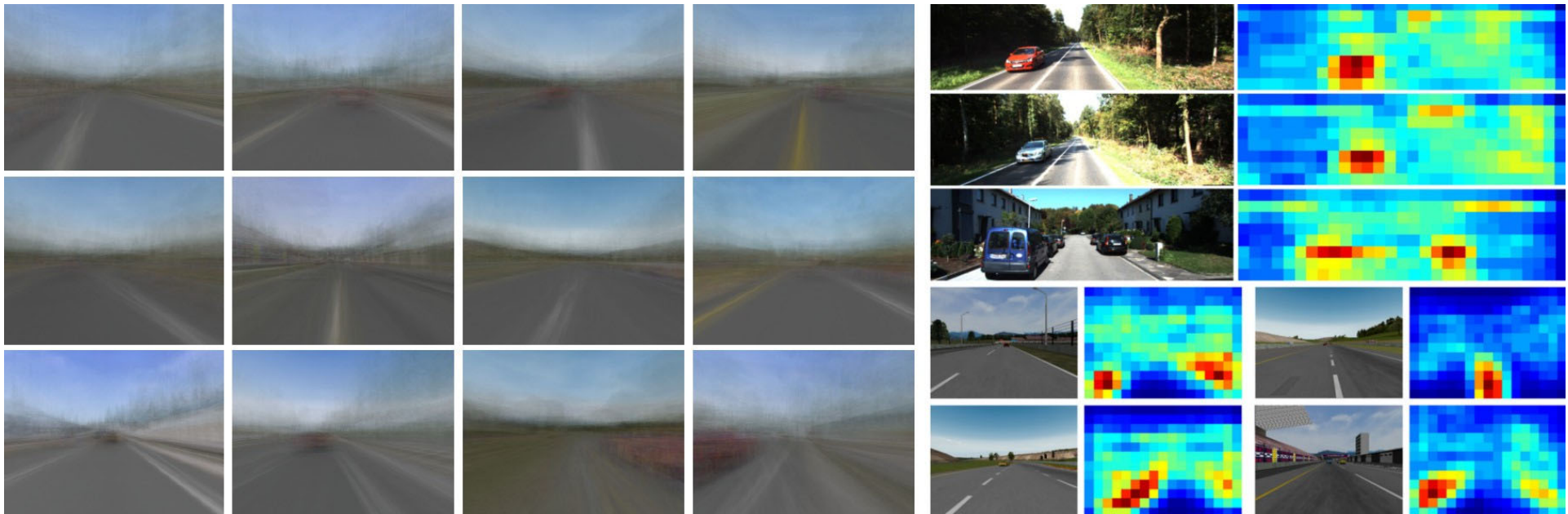
- **Attributes** of the environment which **limit space of actions** [Gibson, 1966]
- In this case: 13 affordances

Direct Perception for Autonomous Driving



- TORCS Simulator: Open source car racing game simulator
- Network: AlexNet (5 conv layers, 4 fully conn. layers), 73 output neurons
- Training: Affordance indicators trained with 2 loss

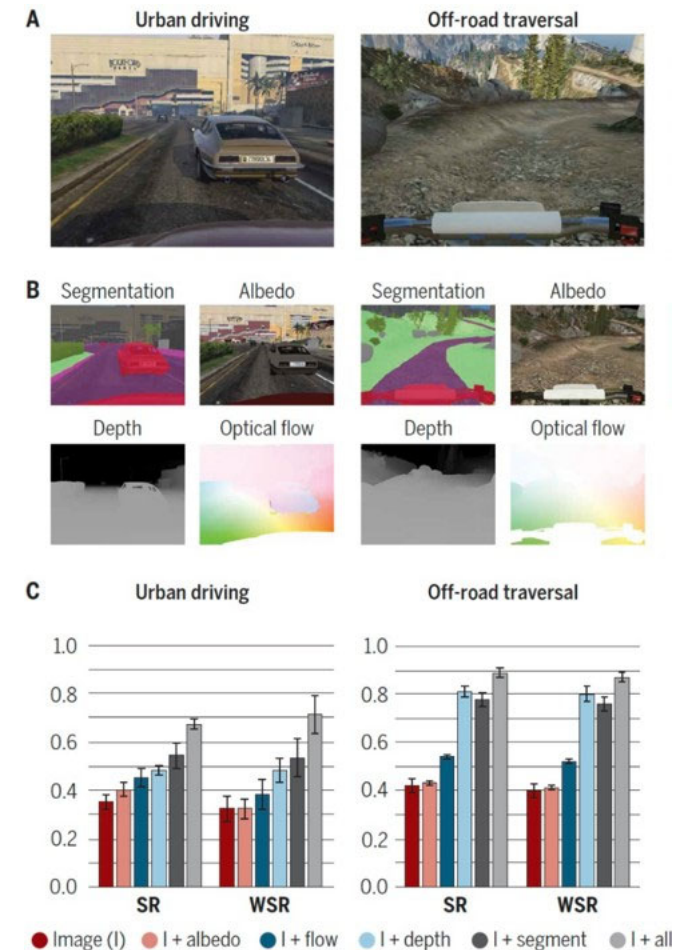
Network Visualization of the Perception Network



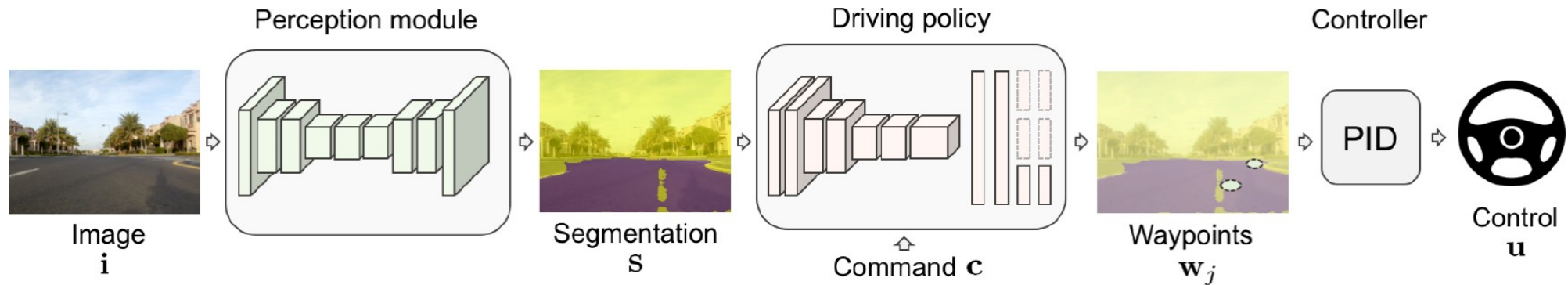
- Left: Averaged top 100 images activating a neuron in first fully connected layer
- Right: Maximal response of 4th conv. layer (note: focus on cars and markings)

Visual Abstractions as Generic Intermediate Representation

- Various visual abstraction as intermediate representation:
 - Segmentation, depth, normals, flow, albedo ...
- Intermediate visual representations improve results
- Consistent gains across simulations / tasks
- Depth and semantic provide largest gains
- Better generalization performance
- Good visual abstraction: invariant, universal, data / label efficient
- Semantic segmentation:
 - Encodes task-relevant knowledge (e.g. road is drivable) and priors (e.g. grouping)
 - Can be processed with standard policy networks



Driving Policy Transfer



Problem:

- Driving policies learned in simulation often do not transfer well to the real world

Idea

- Encapsulate driving policy such that it is not directly exposed to raw perceptual input or low-level control (input: semantic segmentation, output: waypoints)
- Allows for transferring driving policy without retraining or finetuning

Autonomous Driving: Reinforcement Learning

Reinforcement Learning

- **So far:**

- Supervised learning, lots of expert demonstrations required
- Use of auxiliary, short-term loss functions
 - Imitation learning: per-frame loss on action
 - Direct perception: per-frame loss on affordance indicators

- **Can we:**

- Learning of models based on the loss that we actually care about, e.g.:
 - Minimize time to target location
 - Minimize number of collisions
 - Minimize risk
 - Maximize comfort
 - etc.

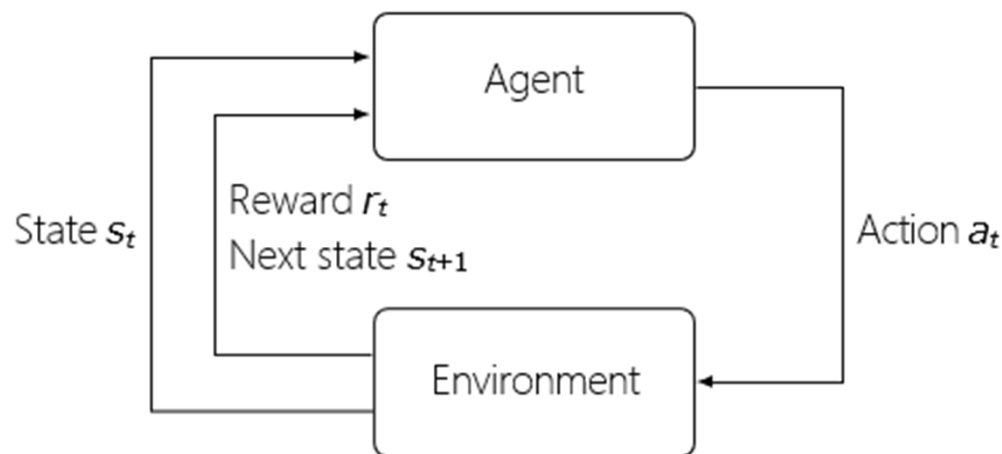
What is Reinforcement Learning?

■ Learning Methods

- The focus of learning is acquiring skills or knowledge from experience. The three well-known and widely studied learning problems in research and practice are:
 - **Supervised Learning:** to learn a mapping between input examples and the target variable. Examples are classification and regression.
 - **Unsupervised Learning:** to learn a model to describe or extract relationships in data. Examples are clustering, feature learning.
 - **Reinforcement Learning:** an agent in an environment learns how to best operate using exploration and feedback.
- Reinforcement learning learns what to do — how to map situations to actions—so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them.
 - An example of a reinforcement problem is playing a game where the agent has the goal of getting a high score and can make moves in the game and receive feedback in terms of punishments or rewards.
 - There is no fixed training dataset, rather a goal or set of goals that an agent is required to achieve, actions it may perform, and feedback about performance toward the goal.

Reinforcement Learning Basics

- In a typical Reinforcement Learning (RL) problem, there is a learner and a decision maker called **agent** and the surrounding with which it interacts is called **environment**. The environment, in return, provides **rewards** and a new **state** based on the **actions** of the agent.
- So, in reinforcement learning, we do not teach an agent how it should do something but present it with rewards whether positive or negative based on its actions.



Agent : Software programs that make intelligent decisions and they are the learners in RL. These agents interact with the environment by actions and receive rewards based on these actions.

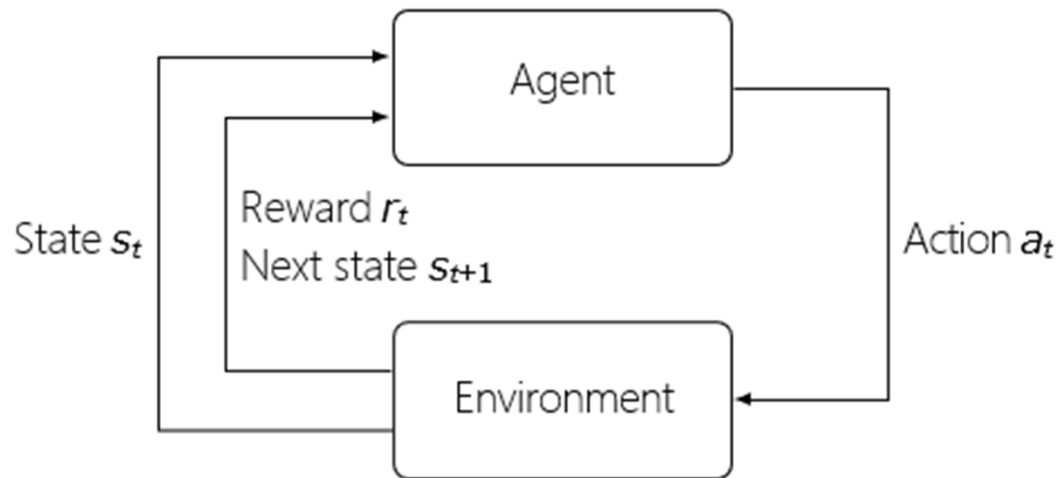
Environment: It is the demonstration of the problem to be solved. We can have a real-world environment or a simulated environment with which our agent will interact.

State: the position of the agent at a specific time-step in the environment. Whenever an agent performs an action, the environment **transits** to a new state depending on the current state and the action.

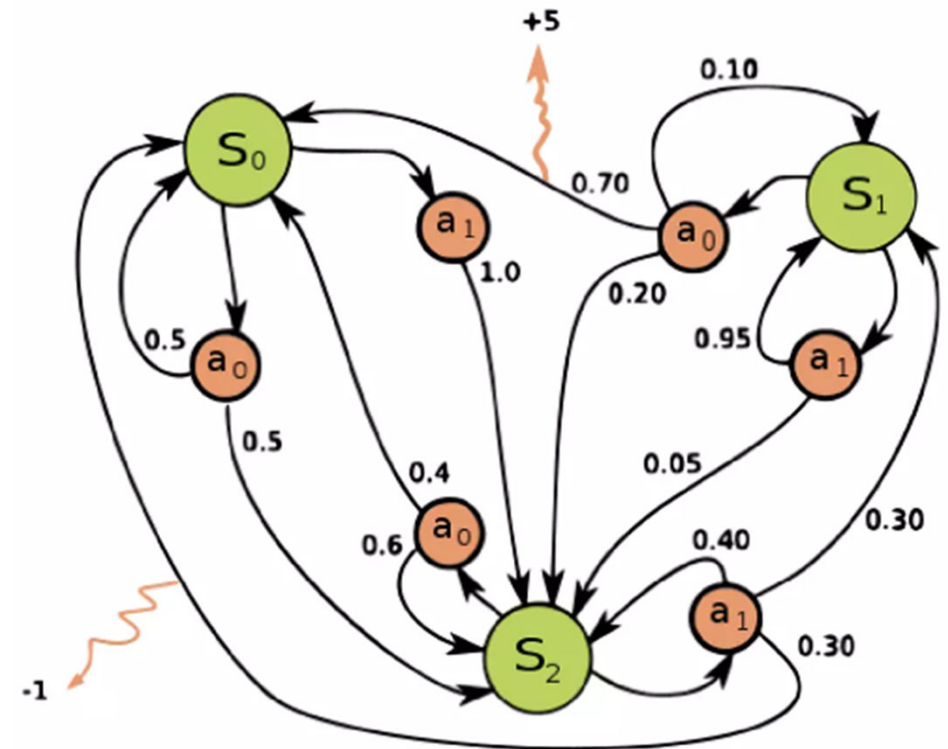
Rewards: numerical values that the agent receives on performing an action at a state in the environment.

Return: sum of cumulative reward received from the environment.

The Agent Environment Interaction



- Agent observes environment state s_t at time t
- Agent sends action a_t at time t to the environment
- Environment returns the reward r_t and its new state s_{t+1} to the agent
- The interaction continues and consecutive actions and state transitions form a trajectory
- Agent wants to learn the best action for each state to maximize the return.



Markov Decision Process (MDP)

- MDP $M = (S, A, P, R, \gamma)$

- S: States

- A: Actions

- P: Transition Probability

$$\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$

- R: Reward

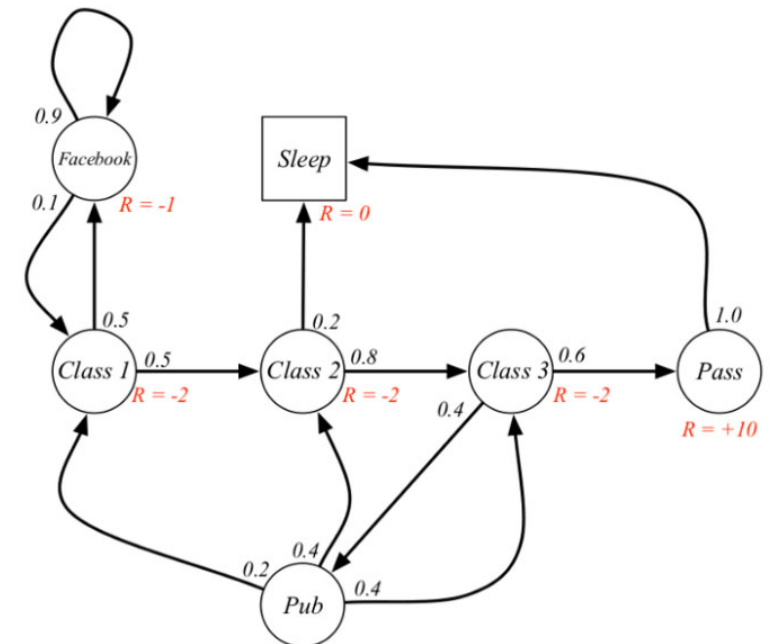
$$\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

- γ : Discount factor

$$\text{Return } G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

- A policy π defines what actions to perform in each state S.

$$\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s]$$



Example: Suppose our start state is *Class 2*, and we follow the state sequence: *Class 2* > *Class 3* > *Pass* > *Sleep*. The expected return with discount factor 0.5 is:

$$-2 + (-2 * 0.5) + 10 * 0.25 + 0 = -0.5$$

Hence the return of state *Class 2* for this sequence state transitions (moves) is -0.5.

Bellman Equations

- How do we discover the actions for a good policy?
 - Try all the possible actions at each state and see what the environment gives back (reward)
- How do we evaluate if the policy is good?
 - Only looking at the immediate reward is not enough, we want the highest expected total return!

given a policy π :

- Bellman Equation:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \quad v_{\pi}(s) \text{ evaluates how good a state is under the current policy } \pi$$

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_{\pi}(s, a) \quad q_{\pi}(s, a) \text{ evaluates how good an action is for the state under the current policy } \pi$$

- Bellman Optimality Equation

$$v_*(s) = \max_{\pi} v_{\pi}(s), \quad q_*(s, a) = \max_{\pi} q_{\pi}(s, a) \quad \Rightarrow \quad v_*(s) = \max_a q_*(s, a)$$

More things to care when training an RL agent ...

- Exploration vs. Exploitation
 - Exploration: trying new action possibilities in order to find better rewards.
 - Exploitation: to keep opting for same actions that has obtained best rewards in the past.
- What if we don't have a model?
 - Model-based: state transition probability and reward distribution are known, full knowledge of the environment
 - Model free: don't know the underlying model, can only learn and update policy based on observations and feedbacks from interactions with the environment
- More efficient way to do exploration
 - On-policy: only learn from experience collected by the most up-to-date policy
 - Off-policy: reused experience from past exploration and experience of others
- ...

Multi-agent Problem

- Have to consider other participants
- Interaction:
 - Cooperative: all agents desire safety and efficiency
 - Competitive: each agent has its own goal
- Environment:
 - Non-deterministic
 - Partially observable
 - Dynamic



Learning in Simulation vs. in Real-world

- Autonomous driving systems require extensive testing to ensure safety and reliability before they can be deployed on public roads. Simulation and physical testing are two complementary approaches used to achieve this.
- Simulation offers scalability and cost-effectiveness, allowing for the testing of numerous scenarios that would be impractical or dangerous to recreate in the real world. It also enables faster iteration and data collection.
 - **Limitations:** Simulators are only as good as the models they use, and there's always a risk that simulated scenarios may not perfectly reflect real-world conditions. It's crucial to validate simulation results with real-world testing.
 - **Examples:** Popular simulation platforms include CARLA and LGSVL.
- Real-world or Physical testing provides real-world validation and helps identify edge cases that may not be accurately represented in simulation.
 - **Limitations:** Real-world testing can be expensive, time-consuming, and potentially dangerous, especially when dealing with complex or rare scenarios. It is also limited by physical constraints and may not be easily scalable.
 - **Examples:** Testing on public roads, proving grounds, and closed-course facilities.

Learning to Drive in a Day

Real-world RL demo by Wayve:

- Deep Deterministic Policy Gradients with Prioritized Experience Replay
- Input: Single monocular image
- Action: Steering and speed
- Reward: Distance traveled without the safety driver taking control (requires no maps / localization)
- 4 Conv layers, 2 FC layers
- Only 35 training episodes

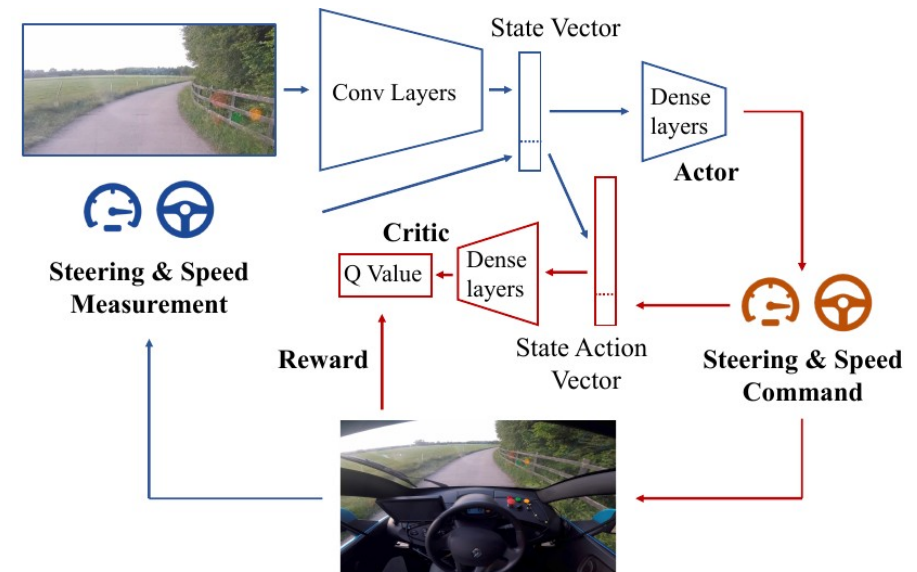
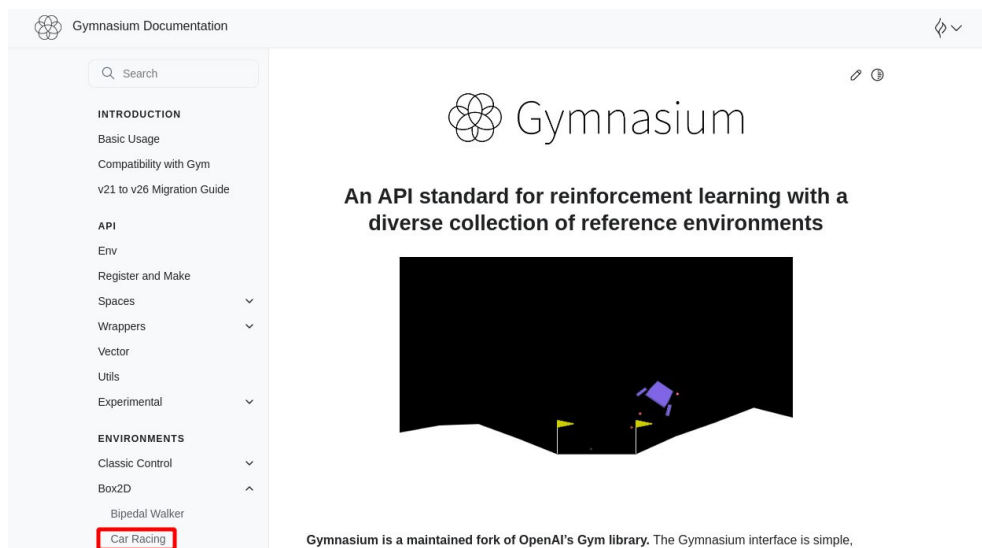


Fig. 1: We design a deep reinforcement learning algorithm for autonomous driving. This figure illustrates the actor-critic algorithm which we use to learn a policy and value function for driving. Our agent maximises the reward of distance travelled before intervention by a safety driver.

Autonomous Driving Simulation in Gymnasium

OpenAI Gym / Gymnasium



Gymnasium Documentation

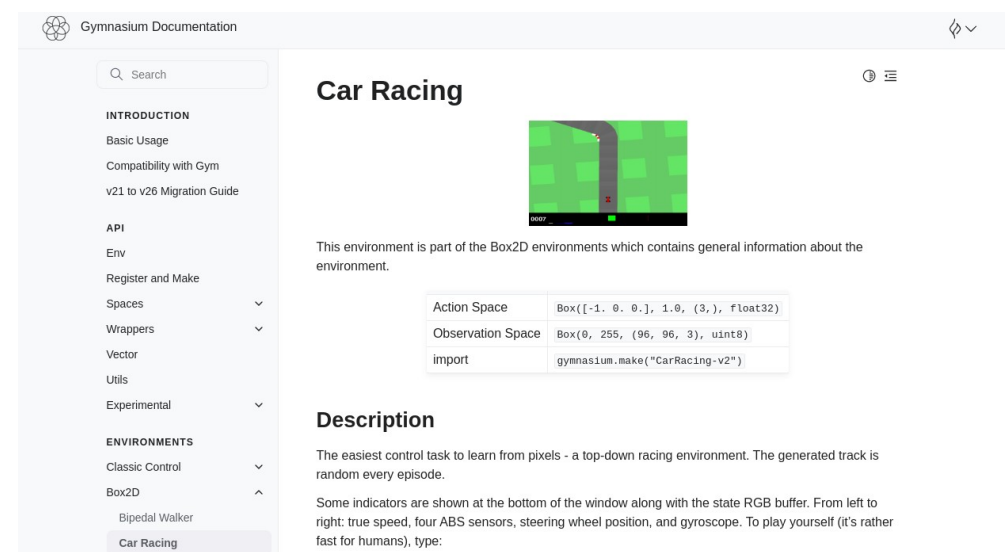
Search

Gymnasium

An API standard for reinforcement learning with a diverse collection of reference environments

Gymnasium is a maintained fork of OpenAI's Gym library. The Gymnasium interface is simple,

Car Racing Environment



Gymnasium Documentation

Search

Car Racing

This environment is part of the Box2D environments which contains general information about the environment.

Action Space	<code>Box([-1.0, 0.], 1.0, (3,), float32)</code>
Observation Space	<code>Box(0, 255, (96, 96, 3), uint8)</code>
import	<code>gymnasium.make("CarRacing-v2")</code>

Description

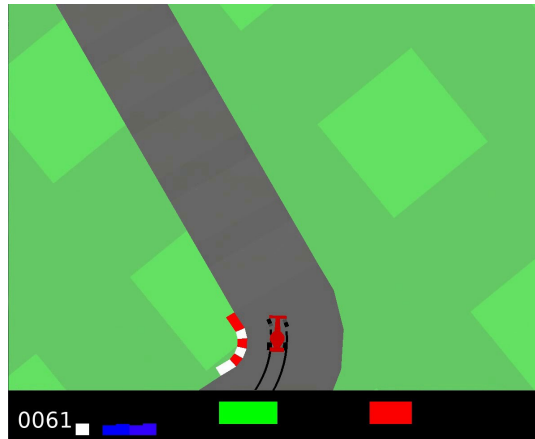
The easiest control task to learn from pixels - a top-down racing environment. The generated track is random every episode.

Some indicators are shown at the bottom of the window along with the state RGB buffer. From left to right: true speed, four ABS sensors, steering wheel position, and gyroscope. To play yourself (it's rather fast for humans), type:

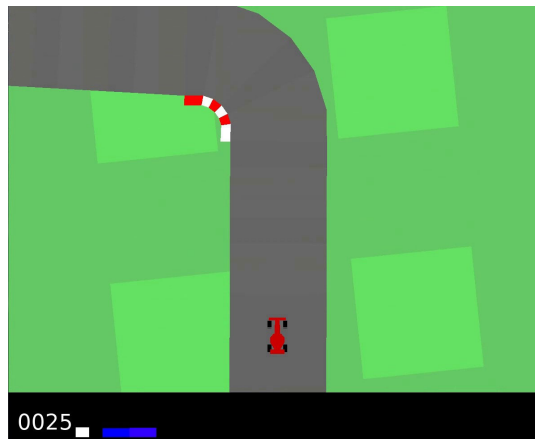
Training a Driving Agent:

Imitation Learning

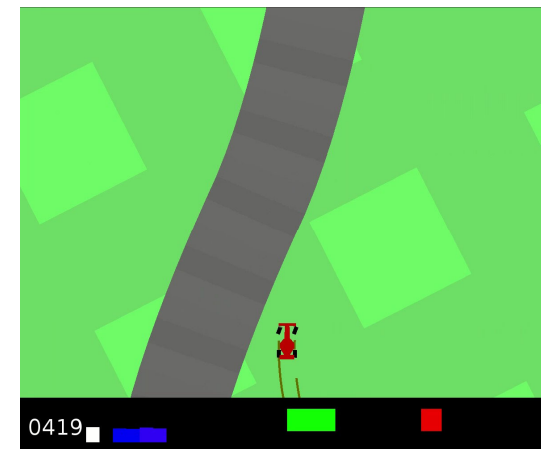
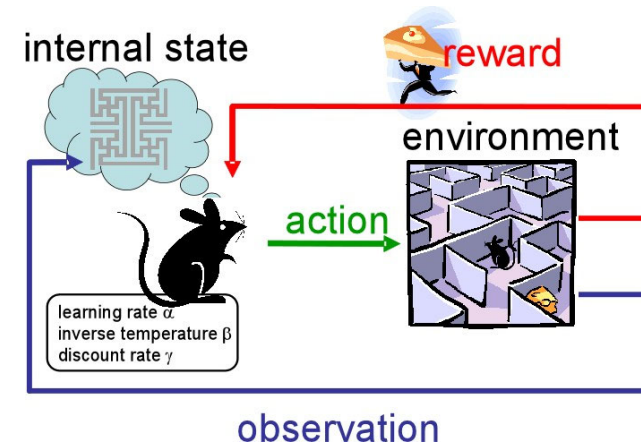
Trainer
(Human Driver)



Trainee
(Neural Network)

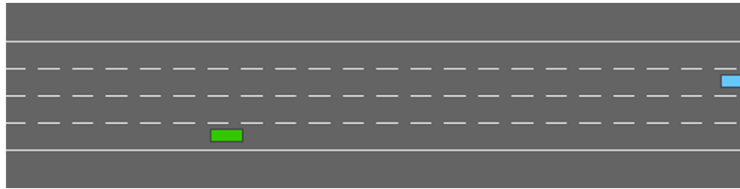


Reinforcement Learning



The highway-env Environments

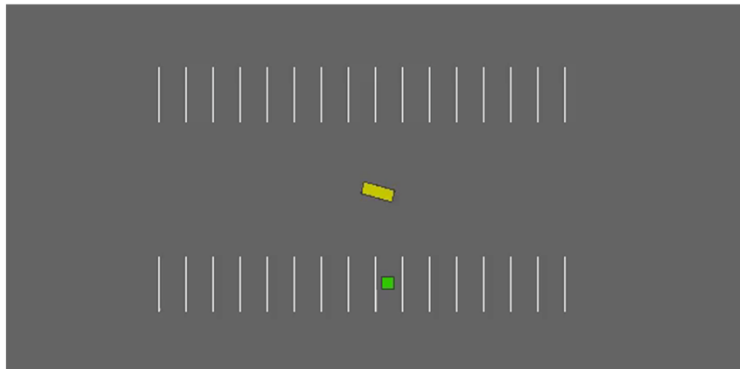
Highway



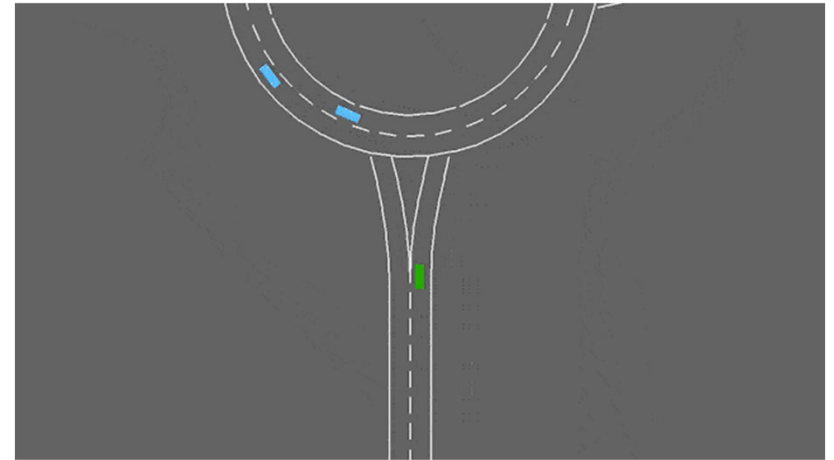
Merge



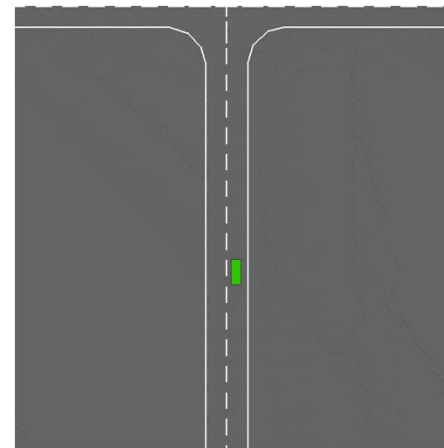
Parking



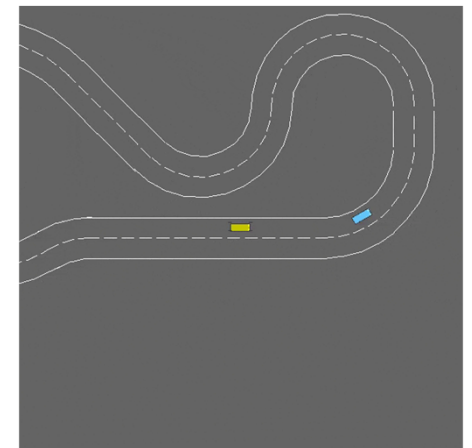
Roundabout



Intersection

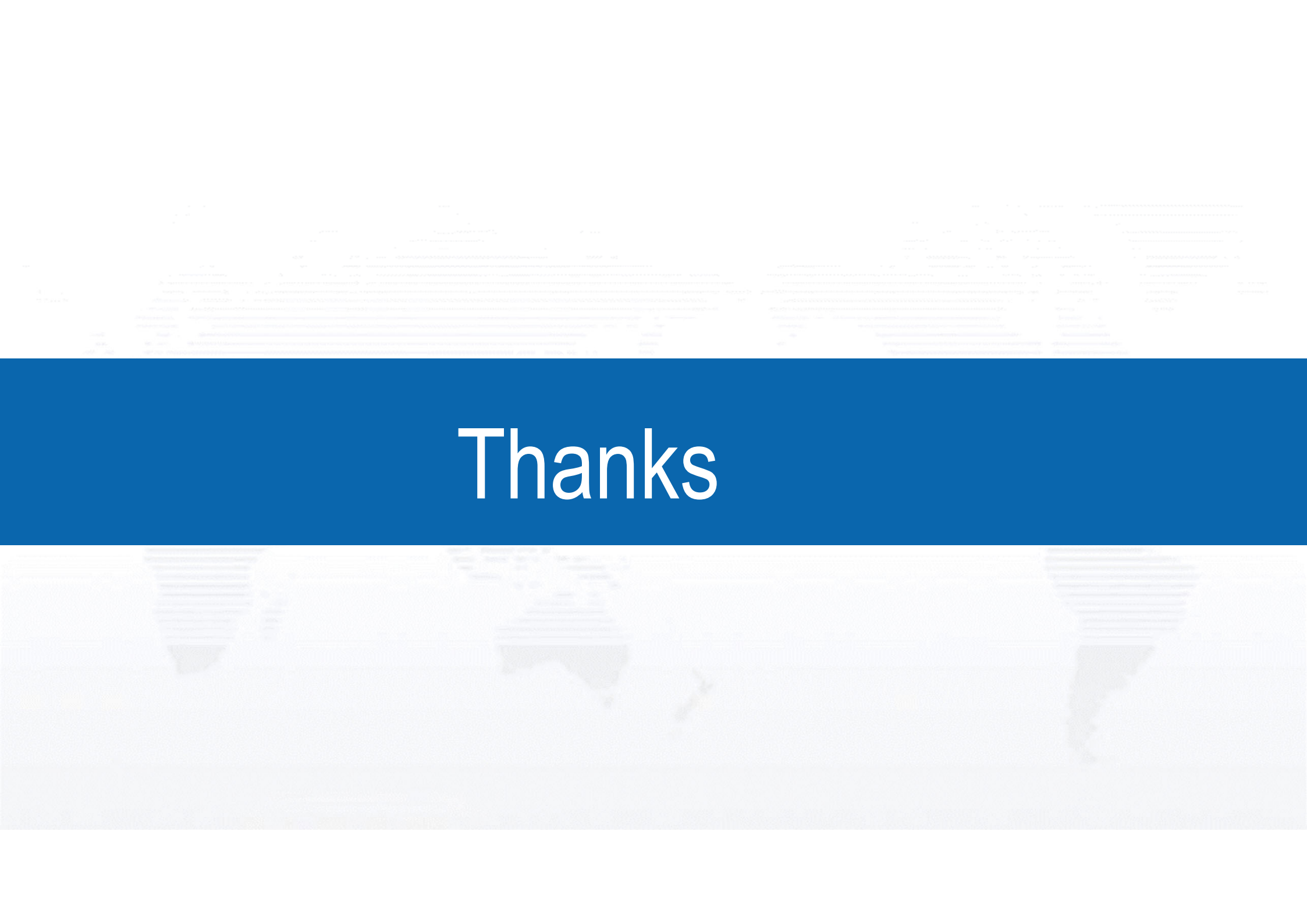


Racetrack



Conclusions

- Autonomous driving will reshape the future of transportation universally
- Technically we are not there yet, still many problems remains
- Artificial Intelligence is one of the main driving forces to make autonomous driving a reality
- There will be a long period of mixed traffic where autonomous vehicles should cope well with human-driven vehicles and other human traffic participants
- There are also legal, ethical, social, security issues to be handled



Thanks